UT San Antonio™

ELEVATE Academy

Evidence-Led Evaluation & AI Tools for Education

UT San Antonio

ELEVATE ACADEMY

ELEVATE (Evidence-Led Evaluation & AI Tools for Education) is supported by the National Science Foundation under Award 2518973, "*Supporting Student Learning and Success in Early Undergraduate Mathematics Courses via AI-Enhanced Education.*" The NSF IUSE: EDU Program supports research and development projects to improve the effectiveness of STEM education for all students.

Juan B. Gutiérrez, Ph.D. Professor of Mathematics, juan.gutierrez3@utsa.edu

The University of Texas at San Antonio

April 22, 2026

Contents

Chapter 1

Introduction

Information

This document is **not** intended to be read from cover to cover. Faculty and staff who are already comfortable with LaTeX, Git, and Python may proceed directly to the course-unit chapters. Those new to one or more of the supporting tools should consult the relevant infrastructure chapter first. Use the routing table on the following pages to identify which chapters apply to your background.

Conventions Used in This Document

Callout boxes. Four types of callout box appear in the text:

- **Information** (teal border): reference material and neutral context.
- **Tip** (blue border): practical guidance and shortcuts.
- **Warning** (magenta border): critical cautions that, if ignored, will cause errors or data loss.
- **Note** (orange border): items requiring attention but not critical.

Code listings. All commands intended to be typed in a terminal are shown in monospace code blocks. Windows commands use a dark background; Linux/macOS commands use a lighter background. Commands that work identically on all platforms use the neutral listing style.
Cross-references. Blue underlined text is a clickable hyperlink. References to chapters, sections, figures, and tables are also clickable in the PDF.

How This Document Is Organized

The document is divided into two parts. The first part covers the technical infrastructure required to work with the curriculum repository: setting up the software

environment, using the command-line interface, writing in LaTeX, managing versions with Git and GitHub, and working with local language-model tools. These chapters are written for faculty and staff who may not have a background in software development; experienced contributors may skim or skip them entirely.

The second part contains the course-unit chapters. Each unit corresponds to one course in the department catalog and documents the course title, student learning outcomes (SLOs), topical outline, and assessment strategy. Units are numbered by their catalog identifier (for example, unit 1023 corresponds to Math 1023). The chapters within each part are independent; a reader interested only in a specific course need not read any other unit chapter.

Chapter Dependencies

Figure 1 shows the dependency structure of the infrastructure chapters. A solid arrow indicates a hard dependency: the target chapter requires the source to be completed first. A dashed arrow indicates recommended order only.

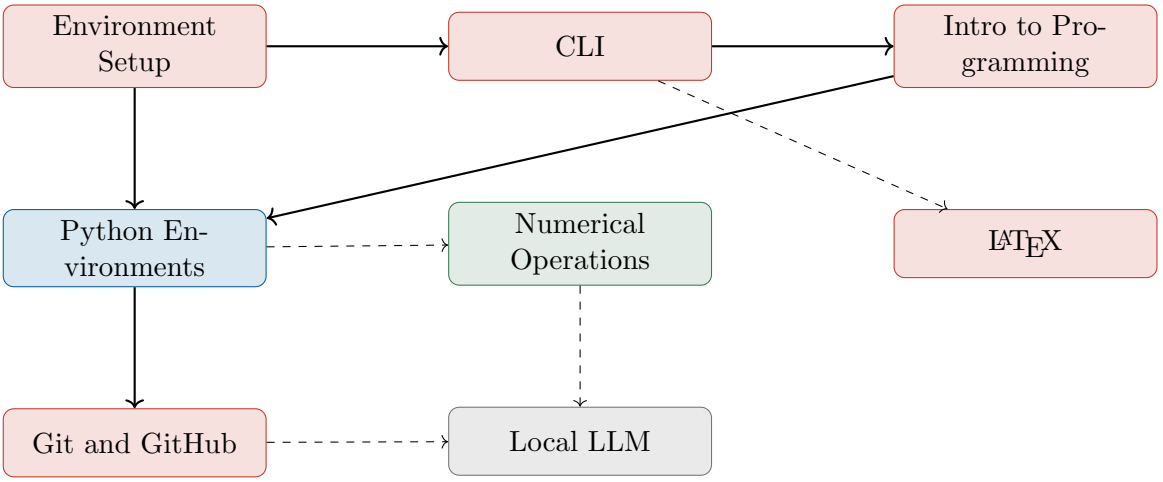


Figure 1: Dependency graph for the infrastructure chapters. Solid arrows indicate hard dependencies; dashed arrows indicate recommended order. Node color encodes category: **Required**, **Essential**, **Desirable**, **Optional**.

Which Chapters Do You Need?

Use Table 1.1 to identify your reading path based on your familiarity with the supporting tools. Chapters marked **Required** must be completed before working with the repository. **Skim** means read the section headings and run the exercises but skip the explanatory prose. **Skip** means the chapter is optional for your profile.

Table 1.1: Recommended reading path by prior experience with the toolchain.

Profile	*Env. Setup*	*CLI*	*Intro Prog.*	*Python Envs.*	*Numerical Ops.*	*LaTeX*	*Git*	*LLM*
New to all tools	•	•	•	•	•	•	•	S
Familiar with LaTeX, new to Python and Git	•	•	•	•	S	–	•	S
Familiar with Python and Git, new to LaTeX	•	S	S	S	–	•	–	S
Experienced with all tools	S	–	–	S	–	–	–	S

• = Required S = Skim – = Skip

Progress Record

The final chapter, Chapter 10 (Assessment and Progress Record), organizes the key completion items from all infrastructure chapters into four categories: Required, Essential, Desirable, and Optional. Open the fillable PDF form in Adobe Acrobat Reader to record and save your responses. Use it throughout onboarding as a readiness checklist.

Estimated Completion Times

Table 1.2 shows estimated completion times per infrastructure chapter. Times assume no prior exposure to the topic; contributors with background in a given area will complete it faster. Course-unit chapters are read as reference material and are not included in this table; reading time for a single unit is approximately 20 to 30 minutes.

Table 1.2: Estimated completion time per infrastructure chapter.

Chapter	Title	Estimated Time
1	Environment Setup	2 hours
2	Command-Line Interfaces and Shell Scripting	2 hours
3	Introduction to Programming Environments	2 hours
4	Python Virtual Environments and Workflow Automation	1.5 hours
5	Numerical Operations with NumPy	2 hours
6	Introduction to LaTeX	1.5 hours
7	Version Control with Git and GitHub	1.5 hours
8	Local LLM Infrastructure with Ollama	1 hour (contributors) / 3 hours (administrators)
9	Assessment and Progress Record	Ongoing
	Total (full path)	**≈ 13.5–15.5 hours**

Chapter 2

Environment Setup

This chapter walks through the installation of all software required for the ELEVATE Academy project. Work through the required sections in order before the first session. Optional sections can be completed at any time. The instructions are written primarily for Windows but include equivalent steps for macOS and Linux throughout.

Familiarity with the command line and with file paths is helpful but not required here. These concepts are covered fully in Chapter 3, which you can read after completing this chapter.

2.1 Required: Install Python

Install any recent Python version from https://www.python.org/downloads/. Go to your downloads and double click on the install file to start installation. By default the Python installer for Windows places its executables in the user's AppData directory, so that it doesn't require administrative permissions. This works for most scenarios. If you're the only user on the system, you might want to place Python in a higher-level directory (e.g. `C:\Python` or `/usr/local/bin`) to have a shorter path to the binaries (sometimes you will need that). Depending on your preferences, either select "*Install Now*" or "*Customized installation*" (my preference). Please make sure you select "*Add python.exe to path*"; this will save you a few headaches later.

In the window "Optional Features" select all features

Select "*Install Python X.XX for all users*" if you can and want. This requires administrative privileges. Select the folder of your choice for the install location.

For PC: If you did not add `python.exe` to the PATH during installation, wait until the installation is complete. Then open File Explorer and right-click on "This PC." Select "Properties" from the context menu. In the System window that opens, click on "Advanced system settings" in the left-hand sidebar. In the new window, ensure the "Advanced" tab is selected and click the "Environment Variables" button near the bottom. Under the "System variables" section, find and double-click on the variable named "Path." This will open a window where

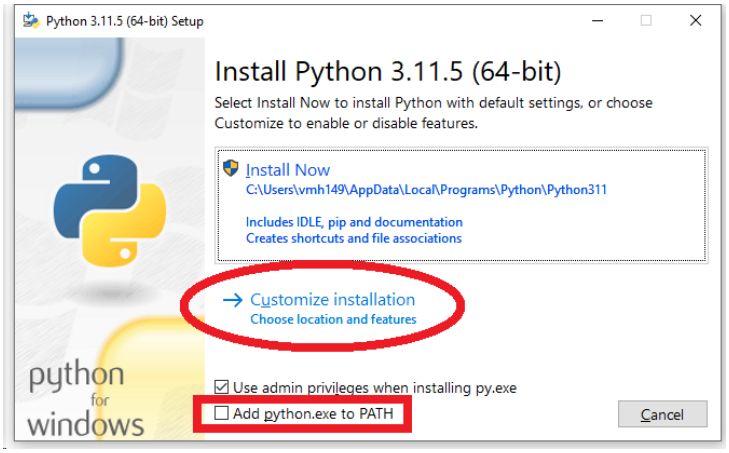


Figure 1: Python installer showing the "Add python.exe to PATH" checkbox at the bottom.

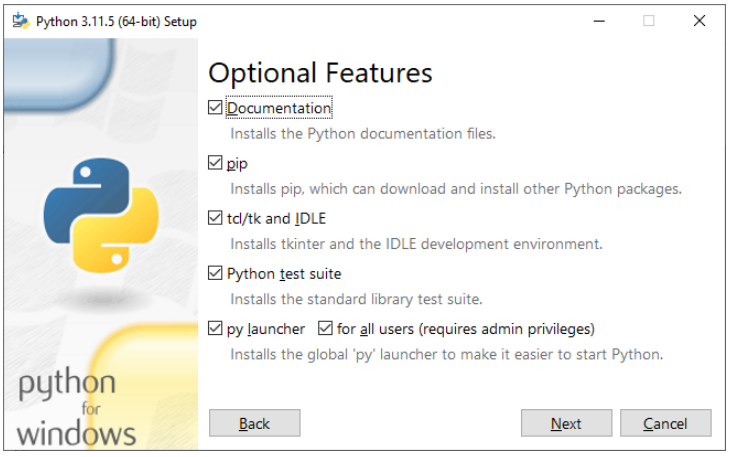


Figure 2: Python Optional Features screen: select all features.

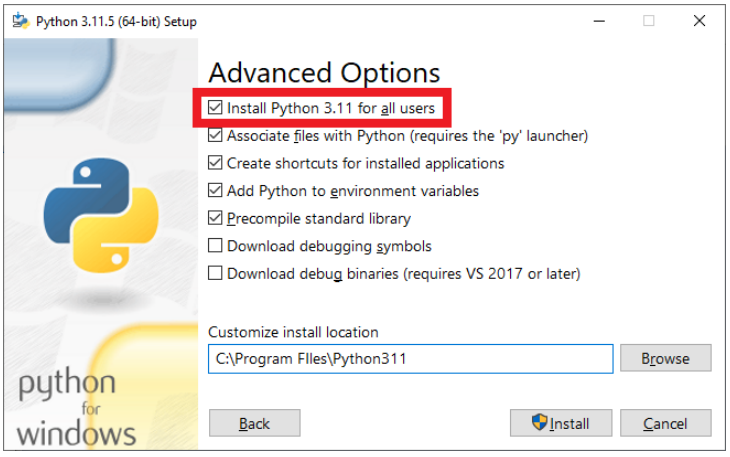


Figure 3: Python Advanced Options screen showing "Install for all users" selected.

you can add the path to the Python executable manually.

For Mac: If you did not ensure Python was added to your system's PATH during installation, you can manually do so using the following steps on a Mac. After installation is complete, open the Terminal application. Enter the command `which python3` or `which python` to find the path to the installed Python binary. Copy that path. Then open your shell configuration file in a text editor. This will typically be `~/.zshrc` if you are using the Zsh shell (default on newer versions of macOS) or `~/.bash_profile` if you are using Bash. Add the line `export PATH="/path/to/python:$PATH"`, replacing `/path/to/python` with the path you copied. Save the file and close the editor. In the Terminal, run `source ~/.zshrc` or `source ~/.bash_profile` depending on the file you edited, so the new PATH is loaded into your environment.

2.2 Optional (but strongly encouraged): Install a LaTeX Distribution

LaTeX is a high-quality typesetting system designed for the creation of technical and scientific documents. Rather than formatting text visually, authors write plain-text source files using a markup language that describes the structure and meaning of the content (such as sections, equations, and references). These source files are then compiled into professionally formatted documents (typically PDFs). LaTeX excels at typesetting equations, managing cross-references, and producing consistent, publication-quality output, making it the standard tool for many fields in science, engineering, and mathematics.

Information

LaTeX is particularly well suited for interaction with Large Language Models

LaTeX is particularly well suited for interaction with Large Language Models (LLMs) because it is a purely textual, declarative markup language that closely resembles structured source code. Unlike binary document formats (e.g., Word or PDF), a LaTeX document exposes its full logical structure—sections, equations, environments, and references—in plain text. This makes it directly interpretable, editable, and generatable by LLMs, which are fundamentally trained on and optimized for text and code. As a result, LLMs can reliably assist with tasks such as drafting mathematical content, refactoring structure, verifying notation, and even debugging compilation issues. In this sense, LaTeX serves as a natural interface between human mathematical reasoning and machine-assisted generation, enabling precise, reproducible, and automatable document workflows.

LaTeX is required if you intend to compile `.tex` documents locally. The program's documents are maintained as LaTeX source files in the repository. For authoring and compiling your own documents, install the distribution for your platform.

Verify the installation on any platform with:

Table 2.1: Recommended LaTeX distributions by platform

Platform	Distribution	Download
Windows	MiKTeX	`https://miktex.org`
macOS	MacTeX	`https://www.tug.org/mactex/`
Linux (Ubuntu/Debian)	TeX Live	`sudo apt install texlive-latex-extra texlive-fonts-recommended latexmk`

```
pdflatex --version
```

Usage of LaTeX for document preparation, including a mathematics notation cheat sheet, is covered in Chapter 7.

2.3 Required: Install Visual Studio Code (VSC)

If you are new to Python, install Visual Studio Code (VSC): There are many options to write Python code, and you could have a favorite one. I must pick one to deliver instruction, and experience has shown me that VSC offers the least amount of friction.

In VSC, there are two main options: "*User Installer*" and "*System Installer*". Use the User Installer if you want to install only for the current user. For all users, use System installer; I prefer this choice because it installs VSC in `C:\Program Files\Microsoft VS Code`.

Visual Studio Code is available for Windows, macOS, and Linux; an identical open-source alternative called VSCodium is available at `https://vscodium.com`.

2.4 Required: Visual Studio Code extensions

Select the Extensions icon on the left and install the following Visual Studio Code libraries

1. **Required**: Python extension for Visual Studio Code.
2. **Optional**: Also install Jupyter (this also installs Pylance, Jupyter Keymap, Jupyter Notebook Renderers, Jupyter SlideShow). Pay attention to the publisher of the extension; use the ones by Microsoft.
3. **Optional**: Install C/C++ for Visual Studio Code, C/C++ Themes & C/C++ Extension Pack.
4. **Optional**: Install the extension "**LaTeX Workshop**" by James Yu. This extension provides syntax highlighting, live preview, and one-click build. Its use is described in Section 7.8.

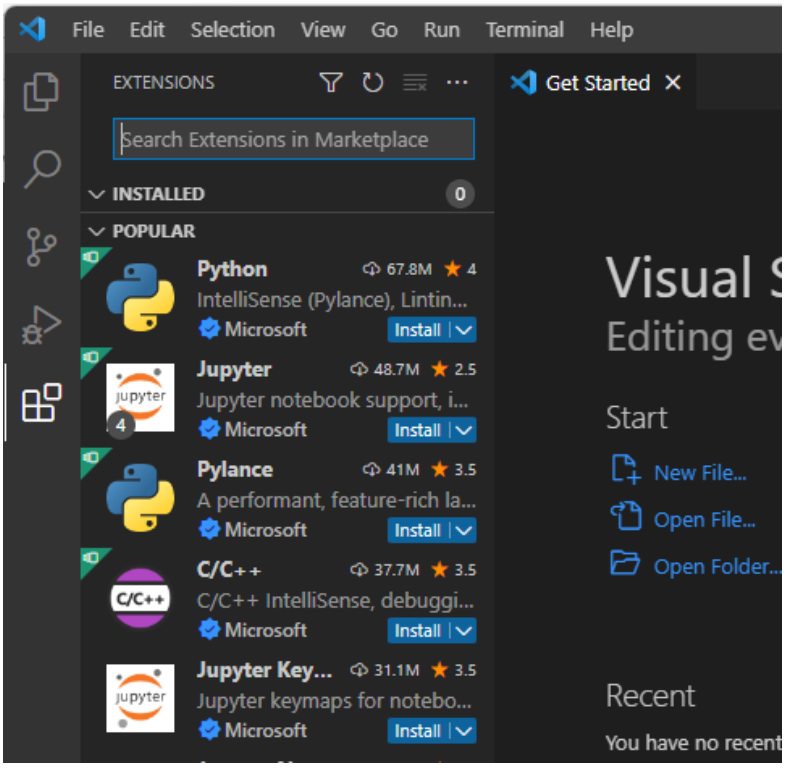


Figure 4: VS Code Extensions marketplace showing the Python and Jupyter extensions by Microsoft.

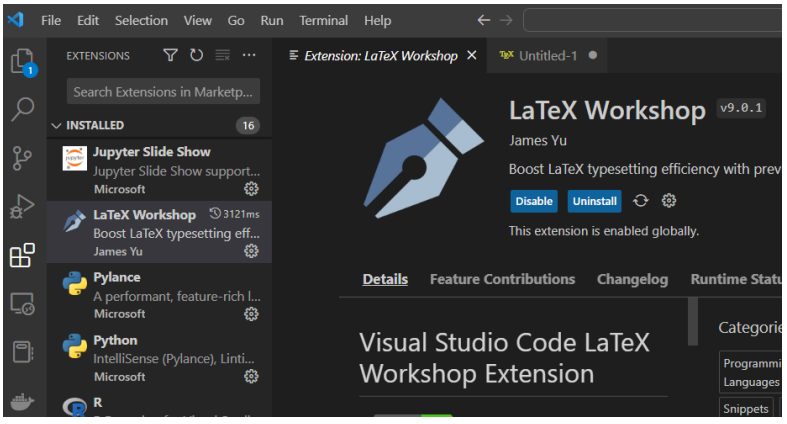


Figure 5: LaTeX Workshop extension page in VS Code.

2.5 Optional: Windows Terminal

For Windows users, install Windows Terminal. This Microsoft app allows you to use a multi document window with tabs for different command consoles like PowerShell, Ubuntu, DOS, Git, etc. You will need this if you want to complete the steps related to Linux in the next section of this document.

2.6 Required: Install Git

Install Git for Windows. For Mac, install Git for Mac. Linux also has a version available. In the second screen, select Visual Studio Code as your Git editor. Use all other default options.

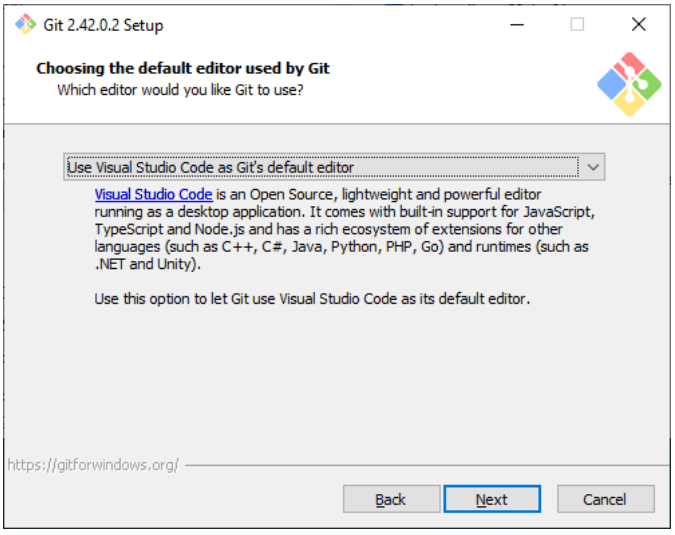


Figure 6: Git setup screen showing Visual Studio Code as the default editor.

2.7 Required: Open Git Bash (or any terminal)

The command line allows you to control your computer by typing instructions, providing precise access to files and system features. For example, typing `ls` on a Unix-like system such as Linux or macOS lists the files and directories in the current location, such as `ls /home/user/Documents` to display the contents of the Documents folder. On Windows, the `dir` command serves the same function, so `dir C:\Users\Alice\Desktop` shows what is on the Desktop. To move between directories, the `cd` command is used; `cd Downloads` changes the working directory to Downloads, while `cd ..` moves one level up. These commands allow efficient navigation and inspection of the file system through the terminal.

Now that Git is installed, clone the ELEVATE Academy repository. Open Git Bash (Windows) or a terminal (macOS/Linux), navigate to the folder where you want to store the project, and run:

```
git clone https://github.com/biomathematicus/ELEVATE.git
```

This creates a folder called `ELEVATE` containing all workshop materials. You only need to do this once. To update your local copy later, navigate into the folder and run `git pull`.

2.8 Required: Open the ELEVATE Academy project in Visual Studio Code

To open the ELEVATE Academy project in VSC, open a terminal, navigate into the `ELEVATE` folder you cloned in the previous section, and run:

```
cd ELEVATE
code .
```

The period after `code` instructs VSC to use the current folder as the working folder. All workshop files will be visible in the VSC file explorer.

2.9 Required: Install Python libraries

Python libraries for the data challenge are installed inside a virtual environment. Complete Chapter 5 to set up your environment, then install the required libraries as described there.

2.10 Optional: Jupyter Notebook

Optional: Now, create a Jupyter Notebook.

1. Abundant relevant information regarding Jupyter in VSC is available at https://code.visualstudio.com/docs/datascience/jupyter-notebooks
2. Activate the environment you want to use (see Chapter 5).
3. You need to attach the kernel of your virtual environment. For example, if I have a virtual environment called venn:

   ```
   python -m ipykernel install --user --name=venn
   ```

4. Open VS Code in your project folder (see Section 2.8).
5. In VSC, run the command `Create:New Jupyter Notebook` from the Command Palette (Ctrl+Shift+P) or by creating a new .ipynb file in your workspace.
6. You might receive a Security Alert regarding the firewall. Allow access.
7. After you enter a command, you might see the following message:

   Install it.

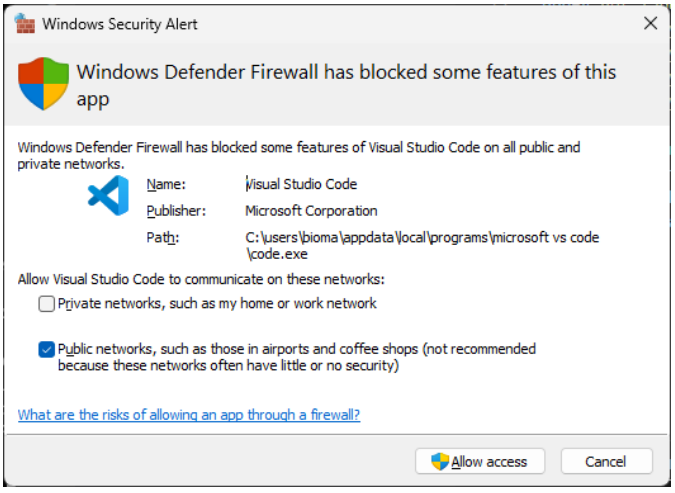


Figure 7: Windows Defender Firewall alert: allow access for Visual Studio Code.

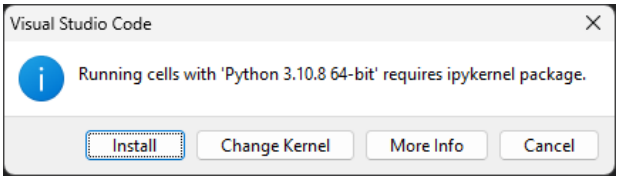


Figure 8: VS Code prompt to install the ipykernel package: click Install.

2.11 Optional: Python commands in Terminal

Finally, you can simply enter Python commands in the Terminal. Open a Command Prompt and type the following command:

```
python
```

This will allow you to enter python commands, e.g. `1+1`

2.12 Troubleshoot

The following errors have been reported by previous participants. If you encounter a different issue, consult the ELEVATE Academy repository issue tracker at `https://github.com/biomathematicus/ELEVATE/issues`.

As reported by Lorena Roa de La Cruz, you could see an error when trying to activate a virtual environment. This is due to configuration of permissions. If this happens, run the command:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

After that, run the script `activate`.

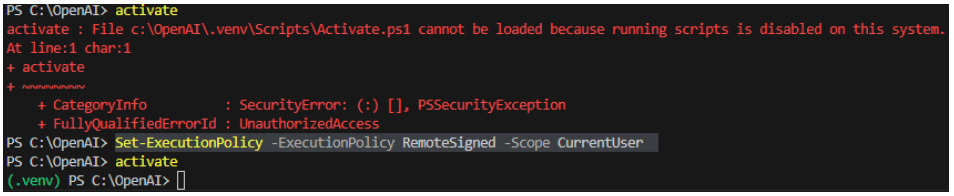


Figure 9: PowerShell execution policy error and fix: running `Set-ExecutionPolicy` followed by `activate`.

2.13 Optional: C/C++ compilers

If you'd like to code in C or C++, you have several options, including GCC, the GNU Compiler Collection, but it requires some effort to install the prerequisites. From Microsoft there is Visual Studio Community Edition (VSCE), which encapsulates the complexity of installing multiple compilers (including the .NET framework SDK). I recommend you install VSCE with all "Workloads" in "Web & Cloud" and "Desktop & Mobile" (except Python).

2.14 Optional: Installing PostgreSQL on Linux

Many Linux distributions ship with older versions of PostgreSQL in their default repositories. To install the latest stable version, it is recommended to use the official PostgreSQL package repository maintained by the PostgreSQL Global Development Group.

The following procedure installs the most recent stable release on Ubuntu or Debian systems.

2.14.1 Install Required Utilities

First install utilities required to retrieve and verify packages.

```
sudo apt update
sudo apt install curl ca-certificates
```

2.14.2 Add the PostgreSQL Repository

Add the official PostgreSQL repository to the system package sources.

```
sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt \
$(lsb_release -cs)-pgdg main" \
> /etc/apt/sources.list.d/pgdg.list'
```

2.14.3 Import the Repository Signing Key

Import the PostgreSQL repository signing key.

```
curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc \
| sudo gpg --dearmor -o /usr/share/keyrings/postgresql.gpg
```

Update the repository configuration to reference the key.

```
sudo nano /etc/apt/sources.list.d/pgdg.list
```

Modify the line to read in a single line (the character \ means line return; it is here just to indicate that all is one line; delete it and join the following three lines of text into a single line):

```
deb [signed-by=/usr/share/keyrings/postgresql.gpg] \
http://apt.postgresql.org/pub/repos/apt \
$(lsb_release -cs)-pgdg main
```

2.14.4 Install PostgreSQL

Update the package index and install the latest PostgreSQL version.

```
sudo apt update
sudo apt install postgresql-16
```

2.14.5 Verify the Installation

Confirm that PostgreSQL was installed successfully.

```
psql --version
```

The output should indicate the installed PostgreSQL version.

2.14.6 Start the Database Server

If the service is not already running, start the PostgreSQL cluster.

```
sudo systemctl start postgresql
```

Verify the server status:

```
sudo systemctl status postgresql
```

2.14.7 Access the PostgreSQL Shell

To open the PostgreSQL interactive terminal:

```
sudo -u postgres psql
```

Exit the shell using:

```
\q
```

Chapter 3

Command-Line Interfaces, Virtual Environments, and Shell Scripting

3.1 What Is a File?

3.1.1 Files and text

A file is a named sequence of bytes stored on a storage device. From the computer's perspective, all files are sequences of bytes. What distinguishes a Python script from a photograph is not the storage mechanism but the interpretation: a text editor interprets bytes as characters; an image viewer interprets them as pixel values. A Python file is plain text: every character you type is stored as a byte, and the Python interpreter reads those bytes as instructions. This is why a Word document cannot serve as a Python program; Word stores invisible formatting bytes alongside the text.

3.1.2 The filesystem: folders and paths

Every file lives inside a folder (also called a directory). Folders can contain other folders, forming a tree. The location of any file can be described by a *path*: a sequence of folder names separated by a delimiter, ending in the filename.

Table 3.1: Path syntax on each operating system.

Operating System	Example path
Windows	`C:\Users\Alice\Documents\hello.py`
macOS	`/Users/alice/Documents/hello.py`
Linux	`/home/alice/Documents/hello.py`

Windows uses backslashes and a drive letter; macOS and Linux use forward slashes and start from a root `/`.

3.1.3 The working directory

At any moment, the CLI is "inside" a particular folder, called the *current working directory* (CWD). Commands that refer to files without a full path are interpreted relative to that folder. When you type `python hello.py`, the interpreter looks for `hello.py` in the CWD. If the file is elsewhere, the command fails. Understanding the CWD is the single most common source of confusion for new CLI users and the root cause of most "file not found" errors.

The CLI provides the commands to navigate the filesystem, inspect its contents, and execute files; these are introduced in the next section.

> *"The command line is the most powerful interface ever invented. It is also the most intimidating. This chapter aims to change that."*

3.2 Introduction: What Is a Command-Line Interface?

When you use a computer, you almost certainly interact with it through a **Graphical User Interface** (GUI): windows, icons, menus, and buttons that you click with a mouse. GUIs are intuitive and visually rich, but they have a fundamental limitation: every action you can take must first be anticipated and implemented as a button or menu item by the software developer. If you need to do something the GUI was not designed for, you are stuck.

A **Command-Line Interface** (CLI) solves this by letting you communicate with the computer in *text*. You type a command; the computer executes it and responds in text. This is not a step backward; it is a different, and in many contexts more powerful, mode of interaction.

3.2.1 A Concrete Analogy

Think of the difference between a restaurant with a fixed menu and a restaurant where you can describe exactly what you want to the chef. A GUI is the fixed menu: fast, friendly, and good enough for most customers. A CLI is talking directly to the chef: it requires more knowledge of ingredients, but the results can be precisely what you need, and you can combine them in ways the menu never anticipated.

3.2.2 Why Developers and Researchers Use the CLI

- **Automation.** A sequence of commands can be saved in a file (called a *script*) and replayed perfectly every time.
- **Precision.** Every option and parameter is explicit and readable.
- **Remote access.** Servers in the cloud rarely have a graphical desktop; the CLI is often the only way in.

- **Speed.** Renaming 10,000 files takes one command; doing it by hand in a GUI might take hours.
- **Reproducibility.** You can share a script with a colleague and know they will get exactly the same result.

3.2.3 The Two Major CLI Worlds

This chapter focuses on two CLI environments:

Windows Command Prompt / PowerShell The native CLI tools in Microsoft Windows. Batch files (`.bat`) and PowerShell scripts (`.ps1`) automate tasks here.

Bash (Linux / macOS) The dominant CLI on Linux systems and macOS. Shell scripts (`.sh`) automate tasks here.

These two worlds use different syntax, different path conventions (backslash \ vs. forward slash /), and different scripting philosophies. A key goal of this chapter is to help you move fluidly between them.

3.2.4 Your First Commands

The best way to demystify the CLI is to try it. Table 3.2 shows equivalent commands in both environments.

Table 3.2: Equivalent CLI commands: Windows vs. Linux/macOS

Task	Windows (CMD)	Linux / macOS (Bash)
Print current directory	`cd`	`pwd`
List files	`dir`	`ls -la`
Change directory	`cd foldername`	`cd foldername`
Go up one level	`cd ..`	`cd ..`
Create a folder	`mkdir myfolder`	`mkdir myfolder`
Delete a file	`del file.txt`	`rm file.txt`
Copy a file	`copy a.txt b.txt`	`cp a.txt b.txt`
Print text to screen	`echo Hello`	`echo Hello`
Clear the screen	`cls`	`clear`

Tip

In both environments, pressing the **Up arrow** key recalls your previous commands, and pressing **Tab** auto-completes file and directory names. These two habits alone will dramatically speed up your work at the CLI.

3.3 Two Worlds: Windows and Linux/Mac

Before writing any scripts, it helps to understand the philosophical differences between how Windows and Linux handle the command line. These differences are not arbitrary; they reflect decades of divergent design history.

3.3.1 Paths and Separators

Windows uses *backslashes* and drive letters to describe file locations:

```
C:\Users\DrG\Documents\ELEVATE Academy\main.py
```

Linux uses *forward slashes* and a single unified file tree rooted at `/`:

```
/home/drg/ELEVATE Academy/main.py
```

3.3.2 Scripts and Executability

On Windows, the file extension determines whether a file is executable. Files ending in `.bat`, `.cmd`, or `.exe` are treated as programs.

On Linux, the extension is irrelevant. Executability is a *permission* stored on the file itself. A file must be explicitly granted execute permission before the system will run it. This is done with the `chmod` command, which we cover in Section 5.3.3.

3.3.3 Environment Variables and PATH

Both systems use an environment variable called `PATH` to find programs. When you type a command, the shell searches each directory listed in `PATH` in order and runs the first match it finds.

- **Windows:** PATH is managed through System Properties → Environment Variables, or via the `setx` command.
- **Linux:** PATH is set in configuration files such as `~/.bashrc` (for the Bash shell) or `~/.zshrc` (for Zsh), which are read every time a new terminal session starts.

3.4 WSL: A Linux Environment Inside Windows

For Windows users who want the power of Linux scripting without leaving their familiar environment, Microsoft provides the **Windows Subsystem for Linux** (WSL). WSL is not an emulator or a virtual machine in the traditional sense; it is a compatibility layer built into Windows that runs a real Linux kernel alongside the Windows kernel.

3.4.1 What WSL Gives You

- A full Bash shell (or Zsh, Fish, etc.)
- A complete Ubuntu (or other distribution) file system
- Access to thousands of Linux command-line tools via `apt`
- The ability to read and write Windows files from Linux (your Windows `C:` drive appears at `/mnt/c/`)
- GPU access for machine learning workloads

3.4.2 Installing WSL

Open PowerShell or CMD *as Administrator* and run:

```
wsl --install
```

Listing 3.1: Installing WSL from PowerShell

This installs WSL 2 and the Ubuntu distribution by default. After a restart, a Ubuntu terminal will open and ask you to create a username and password. That account is independent of your Windows login.

Note

Throughout the remainder of this chapter, all Linux instructions apply equally to WSL on Windows and to a native Linux machine. Treat them as the same environment.

3.4.3 The Elegant Solution: Visual Studio Code and the WSL Extension

One of the most powerful aspects of WSL is its integration with **Visual Studio Code** (VS Code), Microsoft's free code editor. Installing the **WSL extension** in VS Code creates a seamless bridge between your Windows desktop and your Linux environment:

- You open VS Code from within the WSL terminal by typing `code .`
- VS Code runs its interface on Windows but executes all code, terminal commands, extensions, and debuggers *inside Linux*
- The file explorer shows your Linux file system
- The integrated terminal is a Bash shell
- Python environments, linters, and formatters all resolve to their Linux versions

This means you never have to choose between the Linux scripting ecosystem and the Windows desktop experience: you get both simultaneously. From this point forward, when we say "open VS Code," we mean launching it from the WSL terminal with `code .`

Tip

To install the WSL extension, open VS Code, press `Ctrl+Shift+X` to open the Extensions panel, and search for "WSL" (publisher: Microsoft). One click installs it.

Chapter 4

Introduction to Programming Environments

4.1 Before the editor: writing your first program in a text file

A Python program is plain text stored in a file with a `.py` extension. Any tool that saves plain text (not formatted text) can create one. Before using a sophisticated editor, you will create and run a program using the simplest text editor available on your system. This establishes a fundamental idea: the editor is irrelevant; the file is what matters.

Table 4.1: Opening a plain text editor on each operating system.

Operating System	**How to open the editor**	**Critical step before typing**
Windows	Press `Win + R`, type `notepad`, press Enter	In the Save As dialog, set "Save as type" to `All Files (*.*)` before saving; otherwise the file saves as `hello.py.txt`
macOS	Open Spotlight (`Cmd + Space`), type `TextEdit`, press Enter	Go to **Format** > **Make Plain Text** before typing anything; TextEdit defaults to rich text, which corrupts code
Linux	Press `Ctrl + Alt + T` to open a terminal, then type `gedit &` and press Enter	No additional step required; `gedit` saves plain text by default

Exercise P-1: Your first Python program

Type the program below exactly as shown into your text editor. Pay attention to the indentation: the spaces before `print(n)` and `if n == 5:` are not decorative; they are required Python syntax and the program will fail without them. Save the file as `hello.py` in a folder you can find (your Desktop is a good choice). Apply the critical step in Table 4.1 for your operating system before saving.

Word processors are not text editors

Microsoft Word, Google Docs, and LibreOffice Writer save files with invisible formatting markup. A Python file saved from one of these tools will fail to run. Use only the editors listed in Table 4.1 or VSC.

```
# Comments begin with the character #
# Comments are not part fo the program.
# They can be used to introduce visual markers for clarity
# ------------------------------------------------
# This program exemplifies inoput-output (I/O),
# loops, conditionals, and assignment
print("This is programming!")
k = 10
for n in range(k):
    print(n)
    if n == 5:
        print("here")
```

The `#` line is a comment; Python ignores it. Comments exist for humans. `k = 10` is an assignment; it creates a variable named `k` and stores the value 10. `for n in range(k):` is a loop; everything indented below it executes 10 times. `if n == 5:` is a conditional; the indented block below only runs when the condition is true.

The program exists as a file, but it does nothing until it is executed by the Python interpreter. Now that you have completed Chapter 3, you have all the tools needed to run it. Open a terminal, navigate to the folder where you saved `hello.py`, and run:

```
python hello.py
```

On macOS and Linux, you may need to use `python3` instead of `python`. The output should match what you would expect from reading the code.

4.2 Introduction

There is a large number of programming languages. For the purpose of scientific computing, we will distinguish two main families of languages: complied vs. scripting languages. They represent two different paradigms of programming language

design, each with its own advantages and trade-offs. The key difference lies in when and how the source code you write is translated into machine code that the computer can understand and execute.

In the realm of scientific computing, the choice between using a compiled language like C++ and a scripting language like Python often comes down to the specifics of the project at hand, including its complexity, required execution speed, and the need for direct hardware access.

4.2.1 Compiled Languages

Compiled languages are those where the source code is entirely translated into machine code before execution. This translation is done by a compiler, a program that takes the entire source code as an input and generates an executable file. Examples of compiled languages include C++, Java, Rust, Go, and Swift.

Compiled languages offer several important benefits including:

- Performance: Since the code is precompiled into machine code, compiled programs usually run faster and more efficiently than interpreted ones.
- Type checking: Many compiled languages have strict type systems, which catch many type-related errors at compile time, before the code is run.
- Security: The source code is not distributed with the software, which makes it harder for others to reverse engineer the software.

However, compiled languages also have some drawbacks:

- Development speed: The edit-compile-run cycle can be time consuming, particularly for larg software projects.
- Portability: The generated executable is usually specific to a type of processor and operating system. Availability for different operating systems or processors might require specific source code alterations and independent compilations.

Compiled languages like C++ are translated into machine code before execution, making them very fast because they're executed directly by the computer's hardware. This can be a significant advantage when dealing with huge datasets or when computation speed is a critical factor.

However, the syntax of languages like C++ is more complex and it may require more lines of code to achieve the same result as a scripting language. Moreover, C++ lacks many of the built-in data analysis functionalities found in languages such as Python, MATLAB or R, so it's necessary to use external libraries like Eigen or Armadillo for matrix operations and other complex computations.

4.2.2 Scripting Languages

Scripting languages, on the other hand, are typically interpreted from text into machine instructions, line-by-line of code just in time for execution. While this can make scripting languages slower than compiled languages, they're often easier to write and read because of their high-level syntax. Examples of scripting languages include Python, MATLAB, R, JavaScript, Ruby, and PHP.

Scripting languages offer several advantages:

- Ease of use: Scripting languages are usually easier to learn and use. They have less verbose syntax and more built-in functionality.
- Flexibility: Since they are interpreted line-by-line, it's easier to make changes to your code and see their effect immediately without having to recompile.
- Portability: The same script can be run on any system that has the correct interpreter installed, without needing to change the code.

Among the drawbacks of scripting languages there ar issues related to:

- Performance: Interpreted languages are generally slower than compiled languages because of the overhead of interpreting code line-by-line during execution.
- Dependency on the interpreter: The code can only be run on systems that have the correct interpreter installed.

Python is particularly well-suited for biomedical data science because it has numerous powerful libraries for data analysis (like Pandas, NumPy, and SciPy) and visualization (like Matplotlib and Seaborn). In addition, Python's simple, straightforward syntax makes it an excellent choice for quick prototyping and exploratory data analysis.

It must be noted that even with all the benefits of Python, the simplified MATLAB syntax for matrix operations facilitates algorithm development since the source code closely resembles mathematical notation.

4.2.3 Comparative Examples

Let us consider a simple example where we want to multiply two matrices A and B. The language examples are listed below in alphabetical oder by the name of the language.

In C++:

```
#include <Eigen/Dense>
#include <iostream>

int main() {
    Eigen::Matrix2f A;
    A << 1, 2,
         3, 4;
```

```
    Eigen::Matrix2f B;
    B << 5, 6,
         7, 8;
    std::cout << "The product AB is:\n" << A * B;
}
```

../source/IntroProgMatMul.c

In MATLAB:

```
A = [1, 2; 3, 4];
B = [5, 6; 7, 8];
disp('The product AB is:')
disp(A * B)
```

../source/IntroProgMatMul.m

In R:

```
A <- matrix(c(1, 3, 2, 4), nrow=2, ncol=2, byrow = TRUE)
B <- matrix(c(5, 7, 6, 8), nrow=2, ncol=2, byrow = TRUE)

print("The product AB is:")
print(A %*% B)
```

../source/IntroProgMatMul.r

In Python:

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print("The product AB is:\n", np.matmul(A, B))
```

../source/IntroProgMatMul.py

From the above example, it's clear that MATLAB code is much more concise and easier to understand than C++. However, if we were dealing with very large matrices or needed to perform this operation many times in a loop, the C++ code might run faster due to being a compiled language.

4.2.4 What is Python?

Python is an *interpreted high-level* programming language for general-purpose computing.

- *Interpreted* means that a program in Python has to be decoded line by line by another program called the Python interpreter and translated into something a computer can understand.
- *High-level* means that there is a strong abstraction of the elements of the hardware of the computer; for instance, memory does not need to be pre-allocated in Python to create a matrix, whereas the C language (a low-level language) requires it. You can think of the level of the language as how close the instructions are to the metal of the machine.

- The process of language interpretation is called *compilation*, that is, the translation of one computer language into another language. This could happen multiple times until the instructions are reduced to a language called *assembler* or *assembly*; this code matches closely the architecture of the computer executing the program. For instance, Java programs are compiled into a language called Java bytecode, which the Java Virtual Machine translates into assembler.

4.2.5 Python Installation

Refer now to Chapter 2 for complete installation instructions for Python, Visual Studio Code, and all required tools.

Being an interpreted language, it means that Python requires an *interpreter*. This is what you install when you "*install Python*". A Python program is simply a text file that contains instructions the interpreter can understand one line (or block of lines) at a time. Since Python is open source, it has many contributors; this results in a robust ecosystem, but the drawback is that there are numerous software libraries that have to be installed while tracking a complex web of co-dependencies. Hence, there is some difficulty in installing a working Python environment to conduct quantitative analysis.

An easy way to use the Python language is through an *Integrated Development Environment*. There are many IDEs that can execute Python code. Being familiar with more than one environment is important as each has strengths and weaknesses. Furthermore, it is a common occurrence that Python programs appear to fail in one environment but work properly in another; this, of course, is a matter of configuration, but it can be a source of frustration in absence of awareness of the potential sources of conflicts in software. Particularly, you want to know how to use tools frequently used in software development, and you need exposure to the multiple options you will find in academia, government and industry.

4.2.6 Executing Python Commands

Commands in Python are case-sensitive, that is, you must respect the capitalization of instructions as they appear in the documentation. The Python interpreter offers multiple mechanisms to interact with the environment. The most commonly used is the Python command line. You can start it form a command shell. Simply type `python`. The command line promt will change to `>>>`

You can enter Python commands in the command line prompt. Press *Enter* or *return* to execute commands. Like most other programming languages, Python provides mathematical expressions. The building blocks of expressions are variables, numbers, operators, scripts and functions

The basic operations are listed below.

Symbol	Operation
$+$	Addition
$-$	Subtraction
$\star$	Multiplication
$/$	Division
$\star\star$	Power

Some commands are just like you would type things in a calculator. If you want to refer the result of the command, you need to give it a name, or in other words, assign the result to a variable. The following command assigns the result to the variable s. (Note that Python didn't save the true answer 7/12 but rather a decimal approximation.)

```
>>> s = 1/3 + 1/4
```

If you want to know what the variable s contains, simply type its name in the command prompt:

```
>>> s
0.5833333333333333
```

If a command does not fit on one line, use a backslash (\) followed by Return/Enter to indicate that the statement continues on the next line. For example,

```
>>> s = 1 - 1/2 + 1/3 -1/4 + 1/5 - 1/6 + 1/7 \
... :- 1/8 + 1/9 - 1/10 + 1/11 - 1/12
```

Type s and Enter to see the new value of s.

```
>>> s
0.6532106782106782
```

Use the double star($**$) to raise something to an exponent.

```
>>> 89**2
7921
```

Imaginary numbers can be used with the constant j, which stands for $\sqrt{-1}$ by default.

```
>>> (1j)**2
(-1+0j)
```

Hence, you must be careful in using the variable j when dealing with complex numbers. What do you expect from the following operation? Can you explain what you observe? You might need to complete the entire chapter before you can answer this question. Record your explanation as answer #1.

```
>>> j = 10
>>> j = j*(-1)**(0.5)
>>> j
```

Does the result stored in the variable `j` match your expectation? Explain. Record your explanation as answer #2.

Python uses conventional decimal notation, with an optional decimal point and leading plus or minus sign, for numbers. Scientific notation uses the letter e or E to specify a power-of-ten scale factor. Imaginary numbers use i as a suffix. (This course does not use complex numbers, but you might generate errors with one.)

Some examples of legal numbers are: 1, -1, 0.0001, 9.87654321, 1.2345e10, 1.2345*10**10, 1j, -2j, 3e5j.

When assigning names to variables or expressions, it is good practice to make the names useful. For example, in the second command above, we gave the result the name rhoSq to imply "rho squared". Observe that you **CAN NOT** have spaces in variable names (i.e. no spaces in names on the left hand side of the equals sign). Giving useful names to your variables or output will help you keep track of what you are doing.

You can use the up and down arrow keys to easily recall and then edit a command with the left and right arrows. Try it. This is the fastest way to repeat Python commands you might have used in the past.

4.2.7 The VSC Python Editor: Executing A Simple Program

Download the file `lab1SimpleInstruction.py`. Open the file in VSC. The following instructions will appear in the editor:

```
k=0;
for i in range(1,11):
    k=k+1/10;
    print(k)
print(k==1)
```

You can click the *Run file* button on the toolbar (as shwon in Figure 1), or press F5. We call all these actions *to execute a program*. The program `lab1SimpleInstruction.py` adds ten times the number 0.1. The comparison of the number 1 against the sum returns a value of *false*, whereas elementary arithmetic tells us that adding the number 0.1 ten times should result in 1, and the returned value should be *true*. Why is this result false? Record your explanation as answer #3. Once the program is executed, the bottom portion of VSC shows a panel labeled TERMINAL, as well as other tabs labeled PROBLEMS, OUTPUT, and DEBUG CONSOLE, as shown in Figure 2

Note that you can enter commands in the terminal. For example, type `1+1` and hit enter. Try mathematical operations such as square root, or trigonometric functions. What challenges do you encouter? Record your explanation as answer #4.

In the program `lab1SimpleInstruction.py` all lines of code were executed sequentially, one at a time. This is what we call a **script file**. That is, when

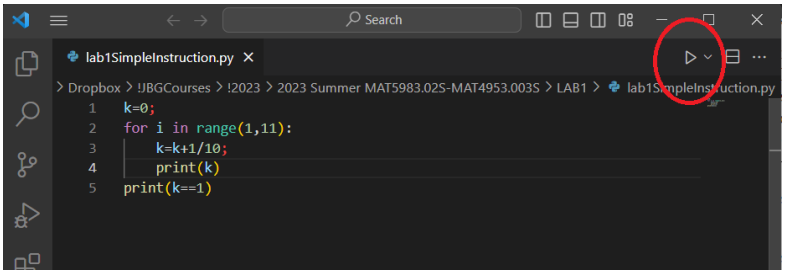


Figure 1: To execute a Python file in VSC, simply click on the Run button, represented by a triangle pointing to the right. It has been highlighted with a circle.

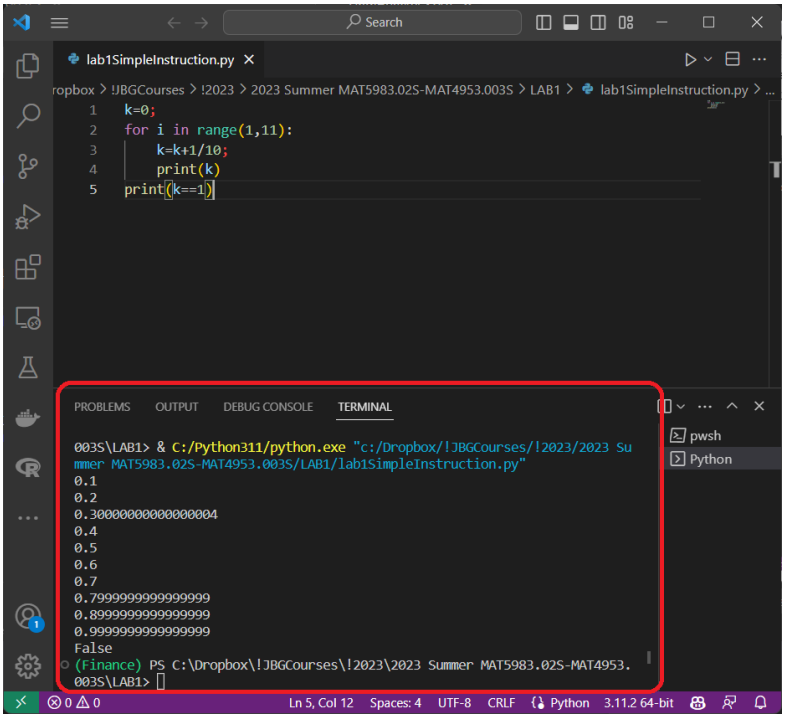


Figure 2: Executing a python program activates the command line. Results are printed in that panel. The command line is highlighted by the round box.

you execute a script file, you expect soemthing tangible to happen, whether it is a message on the screen, a file altereed, communication taking place, etc.

There is a way to use source code files to hold code that will be executed later, without necessarily executing any instruction. At this pointm, open the file `lab1Function.py`. You will see the following instructions:

```
1 def sum1(n):
2     # sum1(n) computes the sum of 1 + 2 + ... + n
3     total = 0;
4     for i in range(1,n):
5         total = total + i
6     return total
```

This file defines a function named `sum1(n)`. The command `sum(3)` returns 6 since $1 + 2 + 3 = 6$. It is instructive to see how this code does its work.

- The instruction `def` indicates the beginning of a *function*. Note that all instructions that are executed inside that function have a vertical alignment

shifted to the right to indicate that all these commands are subordinate to `def`. This shift in vertical alignment is called indentation. It is part of the syntax of the Python programming language, therefore you must always treat indentations strictly.

- The instruction `# sum1(n)...` is a *comment*. Lines of code that start with the symbol `#` are not executed. The semicolon at the end is optional.
- The instruction `total = 0;` is an *assignment*. This line of code creates the variable `total` and assigns to it the value zero.
- The instruction `for i in range(1,n):` starts a *loop*. The subordinate instructions, identified by indentation, will be repeated n times. Particularly, the instruction `range(1,n)` creates a sequence of integers starting in 1 and ending in n. What is the outcome of `range(10,13)` ?
- The instruction `total = total + i` is an assignment. It is very important to emphasize that this is not an equation; if it were, `total` would cancel and `i` would be zero. What takes place is that for every repetition of the loop, the variable `total` is redefined to contain its previous value plus the value of `i`.
- The instruction `return total` is the output of the function. That is, if somehow we could execute this function like `x = sum1(3)`, the variable `x` would contain the value 6, because `sum1(3)` $= 1 + 2 + 3 = 6$.

Now, we will modify this file. Append at the end a program called `sum2` in which we sum integers squared. The next question has to be: How do we execute these functions?

The command `python [filename]` can be used to execute a Python program from the terminal, or any command line prompt. But if we run the following in the command prompt, nothing happens:

```
python lab1Function.py
```

Why? Record your explanation as answer #5.

Now, add the following line of code to the end of the file, **without indentation**:

```
print(sum1(2))
```

Execute the program. What was the result of `sum1(2)`? Is this what you expected? Why? Record your explanation as answer #6. Compute by hand what the output of `sum1(3)` should be. Now change the last line in the program to read `print(sum1(2))`

What we did was simply to define a function and then use it. Note that if you place the previous line of code at the beginning of the file, the Python interpreter would raise an exception because the function `sum1` is not known to the interpreter yet... keep in mind that the Python interpreter executes one line of code at a time

in sequential order (perhaps with branching and repetition, as we will explore later). Therefore, functions must be defined before they can be invoked within a file.

Did the progeam prodice the result you were expecting? In case it did not, enter the following instruction in the terminal: `man range`. This command shows the manual (`man`) for the function `range`. The fact that `range` behaves in certain ways does not force you to abide by that particular design. In fact, you can modify `sum1` so that the outcome corresponds to what a naive user would expect. To achieve this, you will need to change the limit of range inside the function. Implement it and explain. Record your explanation as answer #7.

4.2.8 Executing Functions

In the previous section, we added a single line of code at the end of the program to compute the function we programmed. However, there is a more interesting use case: Create a function that other programs can reuse. In order to use an existing function from a file, we must import that function into the Python environment. We already did something like this when we imported NumPy.

We will now import the function `sum1` from the file `lab1Function.py`. To do this, execute the following command:

```
1  import lab1function as LAB1
```

This instruction loads into memory a directory of all functions present in that file. Now you can invoke the functions you created. For example,

```
1  LAB1.sum2(3)
```

It is also possible to load into memory a single function from a file. We can accomplish this by executing

```
1  from lab1function import sum1
```

In this case, there is no alias associated with the function. To use, you can simply call the function with a parameter, as in `sum2(3)`

Do some research on how to load multiple functions with a single instruction. Explain. Record your explanation as answer #8.

4.3 Introduction to NumPy

Numerical operations can be programmed in Python. A library called `NumPy` encapsulates much needed functionality and has become the *de facto* standard. The basic data element in `NumPy` is a multidimensional array. This allows you to solve many technical computing problems, especially those with matrix and vector formulations, in a fraction of the time it would take to write a program in a low-level language such as C.

For example, at the Python prompt type in the following command and press Enter.

```
import numpy as np
```

This command loads into memory a reference to the NumPy library. Now you can access all functions in this library by using the prefix `np`; this is an *alias*. You can pick any alias you want, but it is customary to use `np` for NumPy. Now enter the following command:

```
np.sqrt(9)
```

The function `sqrt` computes the square root of a real number. Here is an example, and the resulting values.

```
>>> rho = (1+np.sqrt(5))/2
>>> rhoSq = rho**2
>>> a = rhoSq + rho
>>> a
4.23606797749979
```

However, you cannot use the function `sqrt` with negative numbers. What would you have to do to alloy Python to compute the square root of a negative number and return a complex number? Record your explanation as answer #9.

The library NumPy offers easy access to commonly used constants, such as:

Constant	Value
`np.e`	Returns the number e
`np.pi`	Returns the number π
`np.inf`	Returns the number ∞
`np.nan`	NaN, not-a-number

Expressions use the arithmetic operators and precedence rules you already know by now. There are also functions, such as cosine or sine. You surround the input to the function by parenthesis. With NumPy, we can now access commonly used mathematical functions. Among others:

Function	Operation
`np.abs(x)`	Absolute value $\|x\|$
`np.sqrt(x)`	Square root $\sqrt{x}$
`np.exp(x)`	Exponential function e^x
`np.log(x)`	$\log(x)$ where x is positive real number
`np.sin(x)`	$\sin(x)$ where x is assumed to be in radians
`np.cos(x)`	$\cos(x)$ where x is assumed to be in radians

The library NumPy has *many* functions. A comprehensive list can be found at `https://numpy.org/doc/stable/reference/`. Now enter the following command:

```
x = np.random.standard_normal(3)
```

There is no output. Why do you think this is the behavior? Record your explanation as answer #10. On the right-hand side of the Spyder window there

is a tab called *Variable Explorer*. It will show the value of x. Alternatively, you can enter

```
print(x)
```

Alternatively, you can print the result directly without assigning it to a variable. Try

```
print(np.random.standard_normal(3))
```

Why are the results of this last operation different to the values stored in variable x? Record your explanation as answer #11.

4.3.1 Matrix Operations

The basic data element in NumPy is a multi-dimensional array. Special meaning is sometimes attached to 1-by-1 matrices, which are called scalars, and to matrices with only one row or column, which are called vectors. Python has other ways of storing both numeric and non-numeric data, but in the beginning, it is usually best to think of everything as a matrix.

Row vectors are delimited in notation by square brackets. A comma separates individual numbers in a row vector. Multiple row vectors of the same size are separated by commas, and are delimited by square brackets to define a matrix. It is common practice to use a capital letter when naming matrices in order to distinguish them from a single vector, scalar or variable value.

Now, we will create a vector and a matrix to demonstrate simple matrix operations

Function	Operation
`x = np.array([1,2,3,4])`	Creates the row vector $\mathbf{x} = [1, 2, 3, 4]$
`b = np.array([1,2,3])`	Creates the row vector $\mathbf{b} = [1, 2, 3]$
`A = np.array([[1,2,3],[4,5,6],[7,8,9]])`	Creates the matrix $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$
`B = np.ones([2,3])`	Creates the matrix $A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$
`C = np.zeros([2,3])`	Creates the matrix $A = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

The following commands demonstrate basic vector manipulation cases:

Operation	Outcome
`x[-1]`	Returns 4, the last element of x
`x[1]`	Returns 2, the second element of x
`x[-2]`	Returns 3, the second to last element of x

The colon (:) operator is notable. In vector notation, it indicates a range of rows or columns. In Python, the first element of a vector has index 0; the second element has index 1, and so on. The colon operator $a : b$ starts at the row or column of index a and covers up to an index less than b. Thus, $A[0 : 2, 0 : 2]$ returns a matrix composed of rows 1 through 2, and columns 1 through 2 with respect to the matrix A.

The following commands demonstrate basic matrix manipulation cases:

Operation	Outcome
`A[1,2]`	Value of 2^{st} row and 3^{nd} column of A
`A[0:2,0:2]`	Returns $A = \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$
`A[1:3,1:3]`	Returns $A = \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix}$

The matrix A has three rows and three columns. Then, why does $A[1:100, 1:100]$ not produce an error? Record your explanation as answer #12.

The size or dimension of a matrix refers to the number of rows and number of columns a matrix has. The `shape` command returns the number of rows and columns of a matrix. Notice that the result is a vector - a one-dimensional matrix with 2 values where the first is the number of rows and the second value is the number of columns. By accessing each element of this vector, we can access each dimension individually.

Using the matrix A of our previous examples,

```
>>> y = A.shape()
>>> y
(3,3)
>>> y[0]
3
>>> y[1]
3
```

The following example will illustrate a subtlety in copying matrices:

```
>>> D = A

array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
>>> D[0,0] = 9
>>> A

array([[9, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

Note that by updating a value in D, the corresponding value in A was changed. Why? What would you need to do to prevent an update in D to change A? Record your explanation as answer #13.

The basic matrix operations are listed below, using the matrices in the previous examples.

Operation	Outcome
$A.T$	Transpose of A.
$A.conj().T$	Conjugate transpose of A.
$A@B.T$	Matrix multiplication $A \cdot B$.
$A \star A$	Element-wise multiplication
A/A	Element-wise division
$A \star\star 2$	Element-wise exponentiation

NumPy gives an easy way of solving systems of linear equations, Given the matrix A and the vector $\mathbf{b}$ from the previous examples, the linear system $A\mathbf{x} = \mathbf{b}$ can be solved by invoking `np.linalg.solve` as follows

```
>>> x = np.linalg.solve(A,b)
array([-0.23333333,  0.46666667,  0.1       ])
```

It is easy to use Matrix multiplication $A\mathbf{x}$ to check the answer

```
>>> A@x
array([1., 2., 3.])
```

Chapter 5

Python Virtual Environments and Workflow Automation

Before we write any scripts, we need to understand *what* those scripts will manage: Python virtual environments.

The Problem: Package Conflicts. Imagine you have two projects on your computer. Project A uses version 1.5 of a library called `numpy`; Project B requires version 2.0. If you install both versions globally, they will conflict. The last one installed wins, and one of your projects could break.

The Solution: Virtual Environments. A **virtual environment** (or *venv*) is an isolated copy of Python with its own set of installed packages, completely separate from every other environment on your machine. Think of it as a self-contained bubble around your project.

- Each project gets its own environment
- Packages installed in one environment cannot interfere with another
- You can recreate an environment exactly from a list of packages
- Deleting an environment is safe and clean

We will cover two strategies for managing environments: The traditional approach via `pip` and an emerging trend called `uv`.

5.1 Traditional Package Management: `pip`

5.1.0.1 Windows (CMD or PowerShell)

```
:: Navigate to your project folder
cd C:\Projects\ELEVATE Academy

:: Create the environment (Python must be installed
   ↪ globally)
python -m venv C:\penv\ELEVATE Academy
```

Listing 5.1: Creating a venv on Windows

5.1.0.2 Linux / WSL

```
# Navigate to your project folder
cd ~/ELEVATE Academy

# Create the environment
python3 -m venv ~/penv/ELEVATE Academy
```

Listing 5.2: Creating a venv on Linux/WSL

In both cases, `C:\penv\ELEVATE Academy` or `~/penv/ELEVATE Academy` is where the environment will be stored. Keeping all environments in a single folder (`penv` stands for "Python environments") makes them easy to find and manage.

5.1.1 Activating a Virtual Environment

Activating an environment modifies your current shell session so that `python` and `pip` refer to the isolated copies inside the venv, not the global ones.

5.1.1.1 Windows

```
C:\penv\ELEVATE Academy\Scripts\activate.bat
```

Listing 5.3: Activating a venv on Windows

After activation, your prompt will change to show the environment name:

```
(ELEVATE Academy) C:\Projects\ELEVATE Academy>
```

5.1.1.2 Linux / WSL

```
source ~/penv/ELEVATE Academy/bin/activate
```

Listing 5.4: Activating a venv on Linux/WSL

After activation:

```
(ELEVATE Academy) drg@machine:~/ELEVATE Academy$
```

Warning

Note the `source` keyword in the Linux command. This is critical and is explained in detail in Section 5.4.

5.1.2 Installing Packages

Once the environment is active, use `pip` to install packages. The syntax is identical on both platforms:

```
pip install numpy pandas matplotlib openai langchain
```

Listing 5.5: Installing packages with pip

To save a list of all installed packages (so you or a colleague can recreate the environment later):

```
pip freeze > requirements.txt
```

To recreate an environment from such a list:

```
pip install -r requirements.txt
```

5.1.3 Deactivating

To leave the environment and return to the global shell (identical on both platforms):

```
deactivate
```

5.1.4 Installing the ELEVATE Academy Required Libraries

With the ELEVATE Academy environment active, install the required libraries for the data challenge:

```
pip install numpy pandas matplotlib scipy jupyter
```

These five libraries are sufficient for all analysis in the data challenge. Additional libraries required for the LLM chapter (including `openai`, `anthropic`, and `langchain`) are installed separately in Chapter 9.

To verify the installation:

```
python -c "import numpy, pandas, matplotlib, scipy, jupyter; print('OK')"
```

You know your core environment is fully functional when:

- `python --version` returns a version number.
- `code .` opens Visual Studio Code from a terminal.
- `git --version` returns a version number.
- The five required libraries import without errors (see above).

The LLM agent programs (`agentGroq.py`, `agentGPT.py`, `agentClaude.py`) are located at `resources/unit_7/FOO/` in the ELEVATE Academy repository. Running them requires API keys configured in Chapter 9.

5.2 Modern Package Management: `uv`

5.2.1 Why `uv` Matters Today

The tools introduced in the previous section (`python -m venv` for environment creation and `pip` for package installation) are the established, stable foundation of Python package management. They are maintained by the Python Packaging Authority (PyPA) under the umbrella of the Python Software Foundation (PSF), a non-profit organisation with a long-term governance structure and no commercial dependency. Every Python installation includes `pip` by default, and knowing how to use it remains an essential skill regardless of what other tools you adopt.

A newer tool called `uv` has emerged and is rapidly gaining adoption in professional environments. Understanding both tools is important precisely because the landscape is in transition.

Information

A note on governance. `pip` is maintained by the Python Packaging Authority under the Python Software Foundation, a non-profit with a community governance model and no commercial dependency. `uv` is an open-source tool developed and maintained by Astral, a venture-capital funded commercial company. Astral has been an excellent steward of the tool and has contributed meaningfully to the broader Python packaging ecosystem. Nevertheless, its long-term trajectory depends on business outcomes in a way that foundation-backed projects do not. Until the ecosystem stabilises around a clear community standard, knowing both `pip` and `uv` is the prudent professional position.

5.2.2 What `uv` Offers

`uv` is a drop-in replacement for `pip` and `python -m venv`, rewritten in Rust for performance. Its principal advantages over the traditional toolchain are:

- **Speed.** Package resolution and installation are 10–100 times faster than `pip` due to a Rust implementation and aggressive parallel downloading.
- **A central cache.** Downloaded packages are stored once in a global cache and reused across environments. On the same filesystem, `uv` uses hardlinks so that multiple environments sharing a package consume no additional disk space beyond the first copy.
- **Lockfiles.** `uv` produces a `uv.lock` file that records the complete, resolved dependency graph with cryptographic hashes. This is a significant improvement over `pip freeze > requirements.txt`, which records only installed package names and versions without distinguishing direct dependencies from transitive ones, and without hash verification.

- `pyproject.toml` **integration.** `uv` works natively with `pyproject.toml`, the modern standard for declaring project dependencies, replacing the older `requirements.txt` workflow.
- **Python version management.** `uv` can download and manage multiple Python versions independently, removing the need for separate tools such as `pyenv`.

5.2.3 Installing `uv`

5.2.3.1 Windows (PowerShell)

```
powershell -ExecutionPolicy ByPass -c "irm
   ↪ https://astral.sh/uv/install.ps1 | iex"
```

Listing 5.6: Installing uv on Windows

`uv` installs itself to `%USERPROFILE%\.local\bin` and adds itself to your user `PATH` automatically. Close and reopen your terminal, then verify:

```
uv --version
```

5.2.3.2 Linux / WSL

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Listing 5.7: Installing uv on Linux/WSL

`uv` installs to `~/.local/bin`. Reload your shell configuration and verify:

```
source ~/.bashrc
uv --version
```

5.2.4 Creating and Managing Environments with `uv`

The environment structure produced by `uv venv` is identical to a standard `python -m venv` environment. The same `Scripts\activate.bat` (Windows) and `bin/activate` (Linux) scripts are present and activation is unchanged.

5.2.4.1 Creating an Environment

```
uv venv C:\uvenv\ELEVATE Academy --python 3.12
```

Listing 5.8: Creating a uv-managed environment (Windows)

```
uv venv ~/uvenv/ELEVATE Academy --python 3.12
```

Listing 5.9: Creating a uv-managed environment (Linux/WSL)

The `--python` flag specifies the Python version. If that version is not already installed, `uv` will download and install it automatically.

5.2.4.2 Activating an Environment

Activation is identical to a standard venv:

```
C:\uvenv\ELEVATE Academy\Scripts\activate.bat
```

```
source ~/uvenv/ELEVATE Academy/bin/activate
```

5.2.4.3 Installing Packages

Within an activated environment, `uv pip install` replaces `pip install`:

```
uv pip install numpy pandas matplotlib
```

Listing 5.10: Installing packages with uv

5.2.4.4 Declaring Dependencies with `pyproject.toml`

For projects that will be shared or reproduced, `uv` works best with a `pyproject.toml` file in the project root that declares dependencies explicitly:

```
[project]
name = "my-project"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "numpy>=1.26",
    "pandas>=2.2",
    "matplotlib>=3.9",
]
```

Listing 5.11: Minimal `pyproject.toml`

With this file present, a single command installs all dependencies and generates the lockfile:

```
uv sync
```

`uv sync` reads `pyproject.toml`, resolves the full dependency graph, installs all packages, and writes `uv.lock`. Running `uv sync` again on another machine, or after a `git pull` that added new dependencies, reproduces the environment exactly.

Information

Commit `pyproject.toml` and `uv.lock` to your repository. Do not commit the environment folder itself. Add it to `.gitignore`.

5.2.5 `requirements.txt` vs. `uv.lock`

Table 5.1: Comparison of `requirements.txt` and `uv.lock`

Property	`requirements.txt`	`uv.lock`
Generated by	`pip freeze`	`uv sync` / `uv lock`
Records	Installed packages only	Full dependency graph
Distinguishes direct vs. transitive deps	No	Yes
Hash verification	No	Yes
Cross-platform reproducibility	Partial	Yes
Should be committed to git	Yes	Yes

5.3 Level 1: Script Files

Now that we understand virtual environments, we can write scripts to activate them automatically. The goal is to create two scripts:

1. `ActivateEnv`: A reusable script that accepts two parameters (environment name and project directory), activates the environment, and opens VS Code.
2. `ELEVATE Academy`: A project-specific one-liner that calls `ActivateEnv` with the correct parameters for the ELEVATE Academy project.

5.3.1 The Windows Solution: Batch Files

5.3.1.1 ActivateEnv.bat

The Windows version, `ActivateEnv.bat`, is stored in `C:\penv` and placed on the system `PATH`. It accepts two parameters: `%1` (environment name) and `%2` (project directory).

```
@echo off
setlocal

:: Read parameters
set "ENV_NAME=%~1"
set "BASE_DIR=%~2"
echo Starting activation of environment '\%ENV_NAME%'...

:: Validate input
if "%ENV_NAME%"=="" (
    echo [ERROR] Environment name not specified.
    goto :eof
)
```

```
if "%BASE_DIR%"=="" (
    echo [ERROR] Base directory not specified.
    goto :eof
)

:: Check if base directory exists
if not exist "%BASE_DIR%" (
    echo [ERROR] Directory not found: %BASE_DIR%
    goto :eof
)

:: Change to working directory
cd /d "%BASE_DIR%"

:: Define activation script path
set "VENV_ROOT=C:\penv"
set "VENV_ACTIVATE=%VENV_ROOT%\%ENV_NAME%\Scripts\activate.bat"

:: Activate the environment
if exist "%VENV_ACTIVATE%" (
    echo Activating...
    call "%VENV_ACTIVATE%"
    echo Activation complete.
) else (
    echo [ERROR] Not found: %VENV_ACTIVATE%
    goto :eof
)

:: Open VS Code in the project directory
echo Launching VS Code...
code .

endlocal
```

Listing 5.12: `ActivateEnv.bat`: Reusable environment launcher (Windows)

Key Windows-specific elements:

- `@echo off` suppresses the echoing of each command to the screen.
- `setlocal` ensures that variable changes are local to this script.
- `%~1` strips surrounding quotes from the first argument.
- `call` is used to invoke another batch file and return control afterward.
- `goto :eof` jumps to the end of the file (a clean exit).

5.3.1.2 ELEVATE Academy.bat

This file lives in the ELEVATE Academy project directory. It simply calls `ActivateEnv.bat` with the correct arguments:

```
ActivateEnv ELEVATE Academy "C:\Projects\ELEVATE Academy"
```

Listing 5.13: `ELEVATE Academy.bat`: Project launcher (Windows)

Because `C:\penv` is on the system `PATH`, Windows finds `ActivateEnv.bat` automatically. You can run `ELEVATE Academy.bat` by double-clicking it or typing `ELEVATE Academy` in any CMD window.

5.3.2 The Linux Solution: Shell Scripts

5.3.2.1 Understanding Shell Script Syntax

A Linux shell script is a plain text file containing Bash commands. Every script should begin with a **shebang line**, a special first line that tells the system which interpreter to use:

```
#!/bin/bash
```

Parameters passed to a script are accessed as `$1`, `$2`, `$3`, etc. The script's own name is `$0`.

5.3.2.2 ActivateEnv.sh

```
#!/bin/bash
# ActivateEnv.sh
# Usage: source ~/penv/ActivateEnv.sh <ENV_NAME> <PROJECT_DIR>
# Note: Must be *sourced*, not executed. See explanation below.

ENV_NAME="$1"
BASE_DIR="$2"

echo "Starting activation of environment '${ENV_NAME}'..."

# Validate input
if [ -z "$ENV_NAME" ]; then
    echo "[ERROR] Environment name not specified."
    return 1
fi

if [ -z "$BASE_DIR" ]; then
    echo "[ERROR] Base directory not specified."
    return 1
fi

# Check if the project directory exists
if [ ! -d "$BASE_DIR" ]; then
    echo "[ERROR] Directory not found: ${BASE_DIR}"
    return 1
fi

# Change to the project directory
cd "$BASE_DIR" || return 1

# Build the path to the activation script
VENV_ROOT="$HOME/penv"
VENV_ACTIVATE="${VENV_ROOT}/${ENV_NAME}/bin/activate"

```

```
# Activate the environment
if [ -f "$VENV_ACTIVATE" ]; then
    echo "Activating..."
    # shellcheck source=/dev/null
    source "$VENV_ACTIVATE"
    echo "Activation complete."
else
    echo "[ERROR] Not found: ${VENV_ACTIVATE}"
    return 1
fi

# Open VS Code in the project directory (WSL-aware)
echo "Launching VS Code..."
code .
```

Listing 5.14: `ActivateEnv.sh`: Reusable environment launcher (Linux/WSL)

Notice several differences from the Windows version:

- `-z` tests whether a string is empty (zero length).
- `-d` tests whether a path is a directory.
- `-f` tests whether a path is a regular file.
- `$HOME` is the Linux equivalent of `%USERPROFILE%`: it expands to your home directory.
- `return 1` exits the script with an error code (we use `return` rather than `exit` for a reason explained in the next section).

5.3.2.3 ELEVATE Academy.sh

```
#!/bin/bash
# ELEVATE Academy.sh
# Usage: source ~/ELEVATE Academy/ELEVATE Academy.sh
# Launches the ELEVATE Academy project environment.

source ~/penv/ActivateEnv.sh ELEVATE Academy "$HOME/ELEVATE
    Academy"
```

Listing 5.15: `ELEVATE Academy.sh`: Project launcher (Linux/WSL)

5.3.3 Making a Script Executable

On Linux, creating a file does not automatically make it executable. You must grant execute permission using the `chmod` command:

```
chmod +x ~/penv/ActivateEnv.sh
chmod +x ~/ELEVATE Academy/ELEVATE Academy.sh
```

`chmod` stands for *change mode*. The `+x` flag adds execute (`x`) permission. Linux permissions are organized around three roles (owner, group, and others), but for personal scripts, `chmod +x` is almost always all you need.

5.3.4 Adding Your Scripts to PATH

On Windows, you added `C:\penv` to the system `PATH` so that `ActivateEnv.bat` could be found from anywhere. On Linux, you do the same by editing your shell configuration file.

```
# Open your bashrc file in a text editor
nano ~/.bashrc

# Add this line at the bottom:
export PATH="$HOME/penv:$PATH"

# Save and reload the configuration
source ~/.bashrc
```

Listing 5.16: Adding ~/penv to PATH in ~/.bashrc

After this, the shell can find `ActivateEnv.sh` from any directory.

5.3.5 The uv Approach: ActivateUVEnv

The following scripts mirror `ActivateEnv` but target `uv`-managed environments stored in `C:\uvenv` (Windows) or `~/uvenv` (Linux). Store all `uv` environments in `uvenv` rather than `penv` to keep the two toolchains visually distinct on disk.

Two behaviours distinguish `ActivateUVEnv` from its `pip`-based counterpart:

1. **Automatic creation.** If the named environment does not yet exist, `uv venv` creates it before activation. There is no separate setup step for new projects.

2. **Automatic synchronisation.** If a `pyproject.toml` is present in the project directory, `uv sync` runs after activation. Dependencies install themselves after every `git pull` that adds new requirements, so no manual `pip install` is needed.

5.3.5.1 ActivateUVEnv.bat (Windows)

Store this file in `C:\uvenv` and add that folder to the system `PATH`.

```
@echo off
setlocal

set "ENV_NAME=%~1"
set "BASE_DIR=%~2"
set "VENV_ROOT=C:\uvenv"
set "VENV_PATH=%VENV_ROOT%\%ENV_NAME%"
set "VENV_ACTIVATE=%VENV_PATH%\Scripts\activate.bat"

:: Validate inputs
if "%ENV_NAME%"=="" (
    echo [ERROR] Usage: ActivateUVEnv ^<env-name^> ^<project-dir^>
    goto :eof
```

```
)
if "%BASE_DIR%"=="" (
    echo [ERROR] Usage: ActivateUVEnv ^<env-name^> ^<project-dir^>
    goto :eof
)
if not exist "%BASE_DIR%" (
    echo [ERROR] Project directory not found: %BASE_DIR%
    goto :eof
)

:: Create environment if it does not exist yet
if not exist "%VENV_ACTIVATE%" (
    echo [INFO] Environment '%ENV_NAME%' not found. Creating with
        ↪ uv...
    uv venv "%VENV_PATH%" --python 3.12
    if errorlevel 1 (
        echo [ERROR] Failed to create environment.
        goto :eof
    )
)

:: Activate
cd /d "%BASE_DIR%"
call "%VENV_ACTIVATE%"

:: Sync dependencies if pyproject.toml is present
if exist "pyproject.toml" (
    echo [INFO] pyproject.toml found. Syncing dependencies...
    uv sync
)

echo.
echo [OK] Environment '%ENV_NAME%' active.
echo Launching VS Code...
code .

endlocal
```

Listing 5.17: `ActivateUVEnv.bat`: uv environment launcher (Windows)

Usage is identical in form to `ActivateEnv.bat`:

```
ActivateUVEnv lunar "C:\Dropbox\Research\Lunar\python"
ActivateUVEnv forecasting "C:\Projects\forecasting"
```

5.3.5.2 ActivateUVEnv: Linux / WSL Shell Function

Add the following function to `~/.bashrc`. Like its `pip`-based counterpart, it runs in the current shell so that activation and directory changes persist in your session.

```
# ------------------------------------------------------
# ActivateUVEnv: Activate a uv-managed venv and open VS Code
# Usage: ActivateUVEnv <ENV_NAME> <PROJECT_DIR>
# ------------------------------------------------------
ActivateUVEnv() {
```

```
    local ENV_NAME="$1"
    local BASE_DIR="$2"
    local VENV_ROOT="$HOME/uvenv"
    local VENV_PATH="${VENV_ROOT}/${ENV_NAME}"
    local VENV_ACTIVATE="${VENV_PATH}/bin/activate"

    if [ -z "$ENV_NAME" ]; then
        echo "[ERROR] Usage: ActivateUVEnv <env-name>
            ↪ <project-dir>"
        return 1
    fi

    if [ -z "$BASE_DIR" ]; then
        echo "[ERROR] Usage: ActivateUVEnv <env-name>
            ↪ <project-dir>"
        return 1
    fi

    if [ ! -d "$BASE_DIR" ]; then
        echo "[ERROR] Project directory not found: ${BASE_DIR}"
        return 1
    fi

    # Create environment if it does not exist yet
    if [ ! -f "$VENV_ACTIVATE" ]; then
        echo "[INFO] Environment '${ENV_NAME}' not found.
            ↪ Creating with uv..."
        uv venv "$VENV_PATH" --python 3.12 || return 1
    fi

    # Activate
    cd "$BASE_DIR" || return 1
    # shellcheck source=/dev/null
    source "$VENV_ACTIVATE"

    # Sync dependencies if pyproject.toml is present
    if [ -f "pyproject.toml" ]; then
        echo "[INFO] pyproject.toml found. Syncing
            ↪ dependencies..."
        uv sync
    fi

    echo "[OK] Environment '${ENV_NAME}' active."
    echo "Launching VS Code..."
    code .
}
```

Listing 5.18: `ActivateUVEnv()` shell function for ~/.bashrc

Reload `.bashrc` after saving:

```
source ~/.bashrc
```

Then call it from any directory:

```
ActivateUVEnv lunar "$HOME/Research/Lunar/python"
```

5.4 The Critical Difference: Executing vs. Sourcing

This section explains one of the most important, and most misunderstood, concepts in Linux shell scripting, especially for developers coming from Windows.

5.4.1 Child Shells

Every time you run a script with `./script.sh` or just `script.sh`, Linux launches a **child shell**: a new, separate shell process that inherits a copy of your environment, runs the script, and then *terminates*. Any changes the script makes to the environment (activating a venv, changing directory, setting a variable) exist only in that child shell. When the child terminates, those changes disappear. Your original terminal is completely unaffected.

This is why running `./ActivateEnv.sh` would seem to do nothing useful: the venv gets activated inside the child, then the child exits, and your terminal is still pointing at the global Python.

5.4.2 Sourcing: Running in the Current Shell

The `source` command (or its shorthand, a single dot `.`) runs a script *in the current shell*: no child is created. Every command executes as if you had typed it directly at the prompt. This means:

- `source /penv/ELEVATE Academy/bin/activate` activates the venv in *your* terminal.
- `cd` changes *your* working directory.
- Variables set by the script persist in *your* session.

```
# These two are equivalent:
source ActivateEnv.sh ELEVATE Academy ~/ELEVATE Academy
. ActivateEnv.sh ELEVATE Academy ~/ELEVATE Academy

# This does NOT work for venv activation:
./ActivateEnv.sh ELEVATE Academy ~/ELEVATE
    Academy     # child shell; changes are lost
```

5.4.3 The Windows Parallel

Windows does not have this distinction in quite the same way. The `call` command in a batch file runs another `.bat` in the same CMD session and returns, which is why the Windows version works as expected. Linux's default behavior of spawning a child process is actually the *safer* design, as it prevents scripts from accidentally modifying your environment, but it requires you to opt in to the shared-session behavior with `source`.

Note

This is also why `ActivateEnv.sh` uses `return` instead of `exit` for error handling. When a script is sourced, `exit` would close your entire terminal. `return` exits only the script, leaving your session intact.

5.5 Level 2: Shell Functions, The Elegant Linux Approach

Sourcing scripts is powerful, but it requires typing `source` or `.` every time. It also means your launcher script must live somewhere on the `PATH` and the user must remember the exact invocation. Linux offers a more elegant solution: **shell functions**.

5.5.1 What Is a Shell Function?

A shell function is like a script, but it is defined directly inside your shell configuration file (`~/.bashrc`). Because it is part of the shell itself (not a separate file), it automatically runs in the current session without requiring `source`. You call it just by typing its name.

5.5.2 Defining the ActivateEnv Function

Add the following to the bottom of `~/.bashrc`:

```
# ------------------------------------------------------
# ActivateEnv: Activate a Python venv and open VS Code
# Usage: ActivateEnv <ENV_NAME> <PROJECT_DIR>
# ------------------------------------------------------
ActivateEnv() {
    local ENV_NAME="$1"
    local BASE_DIR="$2"

    echo "Starting activation of environment '${ENV_NAME}'..."

    # Validate input
    if [ -z "$ENV_NAME" ]; then
        echo "[ERROR] Environment name not specified."
        return 1
    fi
```

```

    if [ -z "$BASE_DIR" ]; then
        echo "[ERROR] Base directory not specified."
        return 1
    fi

    if [ ! -d "$BASE_DIR" ]; then
        echo "[ERROR] Directory not found: ${BASE_DIR}"
        return 1
    fi

    local VENV_ACTIVATE="$HOME/penv/${ENV_NAME}/bin/activate"

    if [ ! -f "$VENV_ACTIVATE" ]; then
        echo "[ERROR] Virtual environment not found:
            ↪ ${VENV_ACTIVATE}"
        return 1
    fi

    # Change directory, activate, and open VS Code
    cd "$BASE_DIR" || return 1
    source "$VENV_ACTIVATE"
    echo "Environment '${ENV_NAME}' activated."
    code .
}
```

Listing 5.19: Shell function version of `ActivateEnv` in ~/.bashrc

The keyword `local` restricts variables to the function's scope, so they do not pollute your shell session.

5.5.3 Defining the ELEVATE Academy Function

Now add a one-line function that calls `ActivateEnv` for the ELEVATE Academy project:

```
# ------------------------------------------------------
# ELEVATE Academy: Launch the ELEVATE Academy project environment
# Usage: ELEVATE Academy
# ------------------------------------------------------
ELEVATE Academy() {
    ActivateEnv "ELEVATE Academy" "$HOME/ELEVATE Academy"
}
```

Listing 5.20: Project-specific function for ELEVATE Academy in ~/.bashrc

5.5.4 Activating the New Functions

After editing `~/.bashrc`, reload it:

```
source ~/.bashrc
```

Now, from *any directory*, simply type:

```
ELEVATE Academy
```

The shell will change to your ELEVATE Academy project directory, activate the virtual environment, and open VS Code, all in one word, with no `source` prefix required.

5.5.5 Why Windows Has No Clean Equivalent

Windows batch files always run as child processes; there is no `source` command in CMD. PowerShell comes closest: you can define functions in your PowerShell profile (`$PROFILE`), and they behave similarly to Bash functions. However, PowerShell's profile is disabled by default for security reasons and requires changing the execution policy. For Windows users who want this one-command experience natively, configuring PowerShell profiles is the path forward, but it involves more administrative overhead than editing `~/.bashrc`. This is one of the practical reasons many developers on Windows adopt WSL.

5.6 Putting It All Together

Figure 5.2 summarizes the complete workflow on each platform.

Table 5.2: Complete workflow comparison: Windows vs. Linux/WSL

Step	Windows	Linux / WSL
Create venv	`python -m venv C:\penv\ELEVATE Academy`	`python3 -m venv ~/penv/ELEVATE Academy`
Reusable launcher	`ActivateEnv.bat` in `C:\penv`	Function `ActivateEnv()` in `~/.bashrc`
Project launcher	`ELEVATE Academy.bat` in project folder	Function `ELEVATE Academy()` in `~/.bashrc`
Make available globally	Add `C:\penv` to system PATH	Functions in `.bashrc` are always available
Launch project	Type `ELEVATE Academy` in any CMD window	Type `ELEVATE Academy` in any terminal
Result	venv active, VS Code opens in project folder	venv active, VS Code opens in project folder (WSL mode)

5.6.1 The Full WSL Experience

When you type `ELEVATE Academy` in your WSL terminal:

1. Bash looks up `ELEVATE Academy` in its list of known functions.

2. `ELEVATE Academy()` calls `ActivateEnv "ELEVATE Academy" "$HOME/ELEVATE Academy"`.
3. `ActivateEnv()` changes to the project directory.
4. The virtual environment is activated in your current shell.
5. `code .` launches VS Code with the WSL extension, connecting your Windows VS Code interface to your Linux backend.
6. You are now working in a fully configured development environment with one word.

5.7 Quick Reference Cheat Sheet

Table 5.3: Shell scripting and virtual environment cheat sheet

Task	Windows (CMD)	Linux / WSL (Bash)
Create virtual environment	`python -m venv C:\penv\NAME`	`python3 -m venv ~/penv/NAME`
Activate environment	`C:\penv\NAME\Scripts\activate`	`source ~/penv/NAME/bin/activate`
Deactivate environment	`deactivate`	`deactivate`
Install a package	`pip install pkg`	`pip install pkg`
Save package list	`pip freeze > req.txt`	`pip freeze > req.txt`
Restore from list	`pip install -r req.txt`	`pip install -r req.txt`
Make script executable	(not required; extension handles it)	`chmod +x script.sh`
Run a script	`script.bat`	`./script.sh`
Source a script	`call script.bat`	`source script.sh`
Edit shell config	(no equivalent in CMD)	`nano ~/.bashrc`
Reload shell config	(restart CMD)	`source ~/.bashrc`
Open VS Code (WSL)	`code .`	`code .`
Script parameter 1	`%~1`	`$1`
Check if variable empty	`if "%VAR%"==""`	`if [ -z "$VAR" ]`
Check if directory exists	`if exist "DIR\"`	`if [ -d "DIR" ]`
Exit a script/function	`goto :eof`	`return` (in function/-sourced script)

Table 5.4: `uv` quick reference: equivalences with `pip` and `venv`

Task	pip / venv	uv
Install `uv`	(not applicable)	See Section 5.2
Create environment	`python -m venv PATH`	`uv venv PATH --python 3.12`
Activate environment	`source PATH/bin/activate`	`source PATH/bin/activate`
Install a package	`pip install pkg`	`uv pip install pkg`
Install from `pyproject.toml`	`pip install -e .`	`uv sync`
Add a dependency	Edit `requirements.txt` manually	`uv add pkg`
Remove a dependency	Edit `requirements.txt` manually	`uv remove pkg`
Generate lockfile	`pip freeze > req.txt`	`uv lock` (or implicit in `uv sync`)
Install from lockfile	`pip install -r req.txt`	`uv sync`
Launcher script (Windows)	`ActivateEnv.bat`	`ActivateUVEnv.bat`
Launcher function (Linux)	`ActivateEnv()` in `.bashrc`	`ActivateUVEnv()` in `.bashrc`
Environment root	`C:\penv` / `~/penv`	`C:\uvenv` / `~/uvenv`

5.8 Summary

This chapter covered the tools and techniques for managing Python environments and automating project setup across Windows and Linux:

- **Python virtual environments**: isolated, per-project copies of Python that prevent dependency conflicts between projects. Each environment has its own set of installed packages, and deleting one has no effect on any other.
- **The standard `pip` and `venv` toolchain**: the foundation-backed tools included with every Python installation for creating environments, installing packages, and recording dependencies in `requirements.txt`. Knowing this toolchain remains an essential skill regardless of what other tools you adopt.
- `uv`: a modern, high-performance alternative developed by Astral that offers dramatically faster installs, a global package cache, reproducible lockfiles with cryptographic hashes, and native `pyproject.toml` support. Because `uv` automatically synchronises dependencies from a lockfile via `uv sync`, environment reproduction becomes a single command rather than a manual

process.

- **Level 1 scripting**: writing reusable launcher scripts (`ActivateEnv.bat` and `ActivateEnv.sh` for `pip`-managed environments; `ActivateUVEnv.bat` and the `ActivateUVEnv()` shell function for `uv`-managed environments) and project-specific one-liners (`ELEVATE Academy.bat`, `ELEVATE Academy.sh`) that activate the correct environment, change to the project directory, and launch VS Code in a single invocation.
- **Level 2 scripting**: shell functions defined in `~/.bashrc` that provide a one-word launch command from any directory, with no `source` prefix required.
- **The `source` vs. execute distinction**: on Linux, running a script normally spawns a child shell whose environment changes vanish when it exits. The `source` command (or its shorthand `.`) runs the script in the current shell, which is why virtual environment activation requires sourcing. This same distinction explains why `ActivateEnv.sh` uses `return` instead of `exit` for error handling.

The `ELEVATE Academy` command you built in this chapter is a small but meaningful example of a broader principle: the CLI lets you encode your workflow into repeatable, shareable, and improvable scripts. Every tedious setup step you eliminate today is cognitive load freed up for the work that actually matters.

Chapter 6

Numerical Operations

6.1 Bits, Bytes, and Floating Point

Computers have a finite number of ways to represent numbers. That is, they can only approximate a small subset of real numbers. Understanding this is an essential component of conducting numerical operations: You need to understand how and why numerical errors accumulate, and when they interfere with a calculation.

The unit of computing representation is a **bit** (unit represented by b), i.e. a circuit that is either open or closed. Eight bits are grouped into one **byte** (unit represented by B), as shown in Figure 1. This number evolved from computer architectures in the early days of computing, and was first standardized by ISO/IEC 2382-1:1993. In one byte, positions go from 0 to 7, usually represented from right to left. A 1 in the first position represents 2^0, in the second position is 2^1, and so on.

A naive way to represent numbers with a computer is to allocate N decimal digits, for each number. For example:

$$012.345$$

This example represents a 6-digit fixed-point system. This 6-digit fixed-point system results in errors in some sum cases:

$$999.999 + 999.999 = \texttt{OVERFLOW}$$

Also errors occur in multiplication and division

$$013.000/003.000 = 004.333$$

In this case, we lose all digits after the third decimal.

We can allow the decimal point to "float" to gain accuracy with the same number of digits. For example:

$$013.000/003.000 = 4.33333$$

We could even represent larger numbers than 6 digits allowed us before:

$$999.999 \times 999.999 = 9.99998 \times 10^5$$

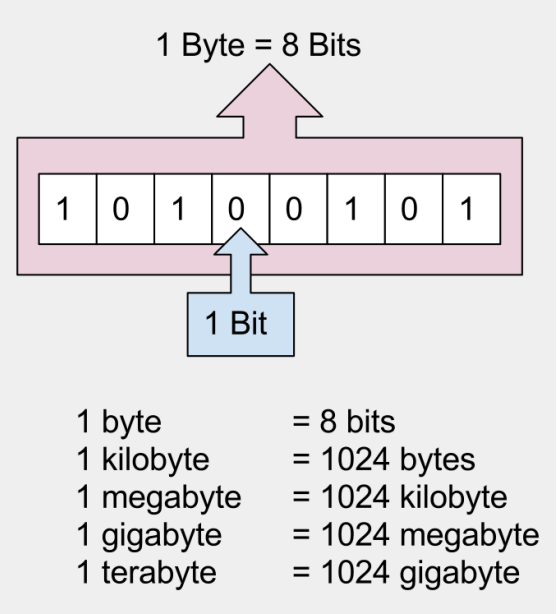


Figure 1: Bits and Bytes [?]. Reading from left to right, the example represents: $2^0+2^2+2^5+2^7 = 1+4+32+128 = 165$. If you read from right to left (this is the standard way to represent bits), you would obtain $2^0+2^2+2^5+2^7 = 165$. In this case, left-to-right and right-to-left produce the same result. But in general they don't have to be the same. What happens if you replace any zero with a one, and compute both directions again? Thus, representation matters, and you need to know the convention.

6.1.1 The IEEE Standard for Floating-Point Arithmetic

The IEEE Standard for Floating-Point Arithmetic (IEEE 754) is a technical standard for floating-point computation established in 1985 by the Institute of Electrical and Electronics Engineers (IEEE).

- Finite numbers may be either base 2 (binary) or base 10 (decimal).
- Each finite number is described by three integers:
  1. s = a sign (zero or one),
  2. c = a significand (or 'coefficient'),
  3. q = an exponent.
- The numerical value of a finite number is $(-1)^s \times c \times bq$ where b is the base (2 or 10), also called radix.

The three fields in an IEEE-754 float. Binary floating-point numbers are stored in a sign-magnitude form where the most significant bit is the *sign* bit, *exponent* is the biased exponent, and *fraction* is the significand minus the most significant bit. Figure 2 shows a 32-bit number, also known as a single-precision floating number, whereas Figure 3 depitcs a 64-bit number, also known as double-precision floating number.

If we want to represent a real number with a finite number N of memory positions, we can reserve one position for the sign, $N-k-1$ for the integer

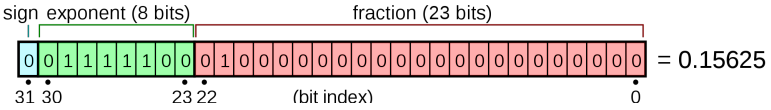


Figure 2: Representation of a single precision number [?]

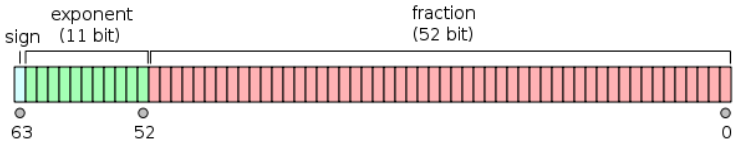


Figure 3: Representation of a double precision number [?].

part, and k for the digits beyond the separator or fractional part. The notation conventionally adopted is [?]:

$$x = (-1)^s[a_{N-2}a_{N-3}...a_k.a_{k-1}a_{k-2}...a_0] = (-1)^s \cdot \beta^{-k} \sum_{j=0}^{N-2} a_j \beta^j \tag{6.1}$$

where β is the base of the representation (it could be any number, but usually it is 2 or 10), s is either 1 or 0, N is the number of digits in the significand, a_j is the j^{th} digit, k is the exponent.

The floating point representation in equation 6.1 limits the minimum or maximum number unless some scaling is allowed. In such case, a *floating point* representation is given by

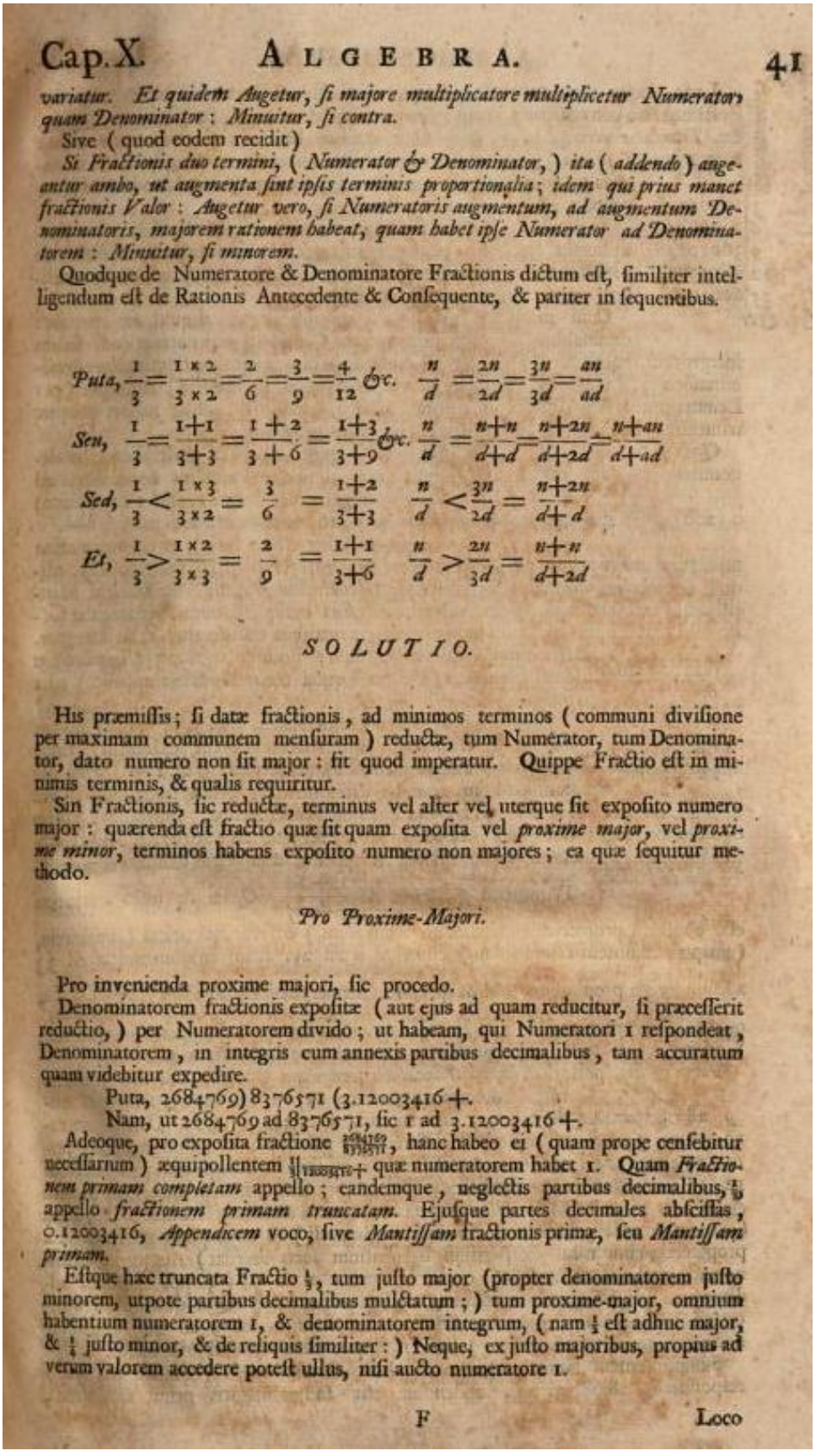
$$x = (-1)^s \cdot [0.a_1a_2...a_t] \cdot \beta^e = (-1)^s \cdot m \cdot \beta^{e-t} \tag{6.2}$$

where $t \in \mathbb{N}$ is the number of significant digits a_i with $0 \leq a_i \leq \beta - 1$, $m = a_1a_2...a_t$ is an integer number called *mantissa* with $0 \leq m \leq \beta^t - 1$ and e is an integer number called *exponent* with $L \leq e \leq U$ (typically $L \leq 0$ and $U \geq 0$). If we require $a_1 \neq 0$ and $m \geq \beta^{t-1}$ then the representation in equation 6.2 is called *normalized*.

The term *mantissa* in computer science commonly refers to the significant digits of a floating-point number, essentially synonymous with *significand*. This usage diverges from its historical mathematical meaning, where *mantissa* denoted the fractional part of a logarithm. Notably, among the many authors that followed John Napier's early logarithmic work, "*Mirifici Logarithmorum Canonis Descriptio*" (1614) [?], it was John Wallis who used the word "*appendage*" in the English version of "*A Treatise of Algebra*," [?, p. 38] (1658), translated into Latin as "manti ∬am" (in modern form *mantissa*) [?, p. 41] (1681) (see Figure 4), to describe the decimal part of a logarithm, not the logarithm of a significand. This distinction highlights the evolution of mathematical terminology over time.

6.1.2 A Bestiary of Floating Point Anomalies

One of the most surprising aspects of floating-point arithmetic is that simple operations do not result in expected results. For instance, the addition $1 - 1/3 - 2/3$



Cap. X. ALGEBRA. 41

variatur. Et quidem Augetur, si majore multiplicatore multiplicetur Numerator, quam Denominator: Minuitur, si contra.

Sive (quod eodem recidit)

Si Fractionis duo termini, (Numerator & Denominator,) ita (addendo) augeantur ambo, ut augmenta sint ipsis terminis proportionalia; idem qui prius manet fractionis Valor: Augetur vero, si Numeratoris augmentum, ad augmentum Denominatoris, majorem rationem habeat, quam habet ipse Numerator ad Denominatorem: Minuitur, si minorem.

Quodque de Numeratore & Denominatore Fractionis dictum est, similiter intelligendum est de Rationis Antecedente & Consequente, & pariter in sequentibus.

Puta, $\frac{1}{3} = \frac{1\times2}{3\times2} = \frac{2}{6} = \frac{3}{9} = \frac{4}{12}$ &c. $\frac{n}{d} = \frac{2n}{2d} = \frac{3n}{3d} = \frac{an}{ad}$

Seu, $\frac{1}{3} = \frac{1+1}{3+3} = \frac{1+2}{3+6} = \frac{1+3}{3+9}$ &c. $\frac{n}{d} = \frac{n+n}{d+d} = \frac{n+2n}{d+2d} = \frac{n+an}{d+ad}$

Sed, $\frac{1}{3} < \frac{1\times3}{3\times2} = \frac{3}{6} = \frac{1+2}{3+3}$ $\frac{n}{d} < \frac{3n}{2d} = \frac{n+2n}{d+d}$

Et, $\frac{1}{3} > \frac{1\times2}{3\times3} = \frac{2}{9} = \frac{1+1}{3+6}$ $\frac{n}{d} > \frac{2n}{3d} = \frac{n+n}{d+2d}$

SOLUTIO.

His præmissis; si datæ fractionis, ad minimos terminos (communi divisione per maximam communem mensuram) reductæ, tum Numerator, tum Denominator, dato numero non sit major: fit quod imperatur. Quippe Fractio est in minimis terminis, & qualis requiritur.

Sin Fractionis, sic reductæ, terminus vel alter vel uterque sit expositio numero major: quærenda est fractio quæ sit quam expositæ vel *proxime major*, vel *proxime minor*, terminos habens expositio numero non majores; ea quæ sequitur methodo.

Pro Proxime-Majori.

Pro invenienda proxime majori, sic procedo.

Denominatorem fractionis expositæ (aut ejus ad quam reducitur, si præcesserit reductio,) per Numeratorem divido; ut habeam, qui Numeratori 1 respondeat, Denominatorem, in integris cum annexis partibus decimalibus, tam accuratum quam videbitur expedire.

Puta, 2684769) 8376571 (3.12003416 +.

Nam, ut 2684769 ad 8376571, sic 1 ad 3.12003416 +.

Adeoque, pro expositæ fractione $\frac{2684769}{8376571}$, hanc habeo ei (quam prope censebitur necessarium) æquipollentem $\frac{1}{3.12003416+}$ quæ numeratorem habet 1. Quam *Fractionem primam completam* appello; eandemque, neglectis partibus decimalibus, $\frac{1}{3}$ appello *fractionem primam truncatam*. Ejusque partes decimales abscissas, 0.12003416, *Appendicem* voco, sive *Mantissam* fractionis primæ, seu *Mantissam primam*.

Estque hæc truncata Fractio $\frac{1}{3}$, tum justo major (propter denominatorem justo minorem, utpote partibus decimalibus mulctatum;) tum proxime-major, omnium habentium numeratorem 1, & denominatorem integrum, (nam $\frac{1}{4}$ est adhuc major, & $\frac{1}{2}$ justo minor, & de reliquis similiter:) Neque, ex justo majoribus, propius ad verum valorem accedere potest ullus, nisi aucto numeratore 1.

F Loco

Figure 4: Page 41 of "*A Treatise of Algebra*," , where the word mantissa is introduced. J. Wallis. *Opera mathematica: De Algebra Tractatus*. Number v. 5 Anno 1685 Anglice editus; Nunc Auctus Latine. Theatrum Sheldonianum, 1693.

results algebraically in zero. However,

$$1 - 1/3 - 2/3 = 1.1102e - 16,$$

$$1 - 2/3 - 1/3 = 5.5511e - 17.$$

Thus, not only the operation is not zero, but also the order of summands matter, i.e. floating point addition is not commutative. Hence, $1 - 1/3 - 2/3 \neq 1 - 2/3 - 1/3 \neq 0$

Because floating point numbers cannot represent all real numbers, errors of representation occur in many cases. But why is the operation noncommutative? Record your explanation as answer #14.

The following instruction compares whether $10^{30} + 1 - 10^{30}$ equals the value 1. Using elementary arithmetic, the answer should be a resounding *yes*. In the Python language, the exponent is represented by a double asterisk, like in Fortran, instead of the carat sign (^) used in most other computing platforms. The double equal sign tests whether the two sides are equal, and returns a Boolean value of *true* or *false*. Hence, the question is translated into the following instruction:

```
10**30 + 1 - 10**30 == 1
```

This particular instruction returns a Boolean value of *true* in Python and *false* in Matlab (the correct Matlab syntax is 10^30 + 1 − 10^30 == 1). However, the following statement:

```
10**30 + 1. - 10**30 == 1
```

returns *false* in Python. Why? Record your explanation as answer #15.

An operation like

```
10**20 + 1 - 10**20 == 1
```

returns different results in Matlab, C, Fortran (*false*) vs. Python (*true*). It might be tempting to declare Python the winner in numerical computation, but what truly happens is that Python does not follow the IEEE-754 standard strictly. To learn about more subtleties of floating point in Python, go to `https://docs.python.org/3/tutorial/floatingpoint.html` In most cases, deviations from the standard in the Python interpreter are harmless and have an ideological origin rather than a technical one. However, it is conceivable that extreme algorithms developed under the IEEE standard would have to be carefully adapted in Python. There are alternatives, but they are beyond the scope of this chapter. For now, it suffices to say that there are landmines in the Python landscape.

We can explore further discrepancies in extreme floating point values by exploring in two numerical environments the results of $10 \cdot 10^b + 1 - 10 \cdot 10^b$ for different values of b. Download and install Python, Matlab, R, or compile a C/C++ program that executes the same operations. Compare what happens in these environments when you choose $b \in \{10, 1e2, 1e10, 1e100, 1e1000\}$. Describe the discrepancies, and analyze. Record your explanation as answer #16.

6.2 Introduction to NumPy

To execute mathematical operations beyond basic operations with real numbers, you need to import a numerical library. Matrix operations are executed through functions in NumPy. Hence, the first step is to load NumPy.

```
In : import numpy as np
```

The library NumPy offers easy access to commonly used constants, such as:

Constant	Value
np.e	Returns the number e
np.pi	Returns the number π
np.inf	Returns the number ∞
np.nan	NaN, not-a-number

With NumPy, we can now access commonly used mathematical functions. Among others:

Function	Operation
np.abs(x)	Absolute value $\lvert x \rvert$
np.sqrt(x)	Square root $\sqrt{x}$
np.exp(x)	Exponential function e^x
np.log(x)	$\log(x)$ where x is positive real number
np.sin(x)	$\sin(x)$ where x is assumed to be in radians
np.cos(x)	$\cos(x)$ where x is assumed to be in radians

Expressions use the arithmetic operators and precedence rules you already know by now. There are also functions, such as cosine or sine. You surround the input to the function by parenthesis. For example, to calculate the square root of two, you type

```
In  : np.sqrt(2)
Out : 1.4142135623730951
```

However, you cannot use the function `sqrt` with negative numbers. What would you have to do to alloy Python to compute the square root of a negative number and return a complex number? Record your explanation as answer #17.

The library NumPy has *many* functions. A comprehensive list can be found at `https://numpy.org/doc/stable/reference/`. You will observe that immediately below the command you entered, the prompt will show `Out[n]: 3.0`, where `n` stands for the command sequence number (recall, the first executed command has $n = 1$, the second $n = 2$, and so on).

Now enter the following command:

```
x = np.random.standard_normal(3)
```

There is no output. Why do you think this is the behavior? Record your explanation as answer #18. On the right-hand side of the Spyder window there

is a tab called *Variable Explorer*. It will show the value of x. Alternatively, you can enter

```
print(x)
```

Alternatively, you can print the result directly without assigning it to a variable. Try

```
print(np.random.standard_normal(3))
```

Why are the results of this last operation different to the values stored in variable x? Record your explanation as answer #19.

6.2.1 Matrix Operations

The basic data element in NumPy is a multi-dimensional array. Special meaning is sometimes attached to 1-by-1 matrices, which are called scalars, and to matrices with only one row or column, which are called vectors. Python has other ways of storing both numeric and non-numeric data, but in the beginning, it is usually best to think of everything as a matrix.

Row vectors are delimited in notation by square brackets. A comma separates individual numbers in a row vector. Multiple row vectors of the same size are separated by commas, and are delimited by square brackets to define a matrix. It is common practice to use a capital letter when naming matrices in order to distinguish them from a single vector, scalar or variable value.

Now, we will create a vector and a matrix to demonstrate simple matrix operations

Function	Operation
$x = np.array([1,2,3,4])$	Creates the row vector $\mathbf{x} = [1,2,3,4]$
$b = np.array([1,2,3])$	Creates the row vector $\mathbf{b} = [1,2,3]$
$A = np.array([[1,2,3],[4,5,6],[7,8,9]])$	Creates the matrix $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$
$B = np.ones([2,3])$	Creates the matrix $A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$
$C = np.zeros([2,3])$	Creates the matrix $A = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

The following commands demonstrate basic vector manipulation cases:

Operation	Outcome
$x[-1]$	Returns 4, the last element of x
$x[1]$	Returns 2, the second element of x
$x[-2]$	Returns 3, the second to last element of x

The colon (:) operator is notable. In vector notation, it indicates a range of rows or columns. In Python, the first element of a vector has index 0; the second element has index 1, and so on. The colon operator $a : b$ starts at the row or column of index a and covers up to an index less than b. Thus, $A[0:2, 0:2]$ returns a matrix

composed of rows 1 through 2, and columns 1 through 2 with respect to the matrix A.

The following commands demonstrate basic matrix manipulation cases:

Operation	Outcome
$A[1,2]$	Value of 2^{st} row and 3^{nd} column of A
$A[0:2,0:2]$	Returns $A = \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$
$A[1:3,1:3]$	Returns $A = \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix}$

The matrix A has three rows and three columns. Then, why does $A[1:100,1:100]$ not produce an error? Record your explanation as answer #20.

The size or dimension of a matrix refers to the number of rows and number of columns a matrix has. The `shape` command returns the number of rows and columns of a matrix. Notice that the result is a vector - a one-dimensional matrix with 2 values where the first is the number of rows and the second value is the number of columns. By accessing each element of this vector, we can access each dimension individually.

Using the matrix A of our previous examples,

```
In  : y = A.shape()
In  : y
Out : (3,3)
In  : y[0]
Out : 3
In  : y[1]
Out : 3
```

The following example will illustrate a subtlety in copying matrices:

```
In  : D = A
Out :
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
In  : D[0,0] = 9
In  : A
Out :
array([[9, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

Note that by updating a value in D, the corresponding value in A was changed. Why? What would you need to do to prevent an update in D to change A? Record your explanation as answer #21.

The basic matrix operations are listed below, using the matrices in the previous examples.

Operation	Outcome
$A.T$	Transpose of A.
$A.conj().T$	Conjugate transpose of A.
$A@B.T$	Matrix multiplication $A \cdot B$.
$A \star A$	Element-wise multiplication
A/A	Element-wise division
$A \star\star 2$	Element-wise exponentiation

NumPy gives an easy way of solving systems of linear equations, Given the matrix A and the vector $\mathbf{b}$ from the previous examples, the linear system $A\mathbf{x} = \mathbf{b}$ can be solved by invoking `np.linalg.solve` as follows

```
In  : x = np.linalg.solve(A,b)
Out : array([-0.23333333,  0.46666667,  0.1       ])
```

It is easy to use Matrix multiplication $A\mathbf{x}$ to check the answer

```
In  : A@x
Out : array([1., 2., 3.])
```

6.3 Specific tasks for this lab

The following chapter is a lecture about Principal Component Analysis (PCA), one of the most basic techniques for data analysis. Implement a principal component analysis according the the following procedure (use NumPy to conduct matrix operations):

1. Select weight at birth plus other numerical variables from the NCHS data set. We will call this the *feature matrix*.

2. Ensure that the mean is zero for every variable. How do you achieve this? Record your explanation as answer #22. We will call this the *normalized feature matrix*.

3. Compute the inner product of the normalized feature matrix. What is the dimension of the resulting matrix? Record your explanation as answer #23.

4. Compute eigenvalues and eigenvectors for the inner product of the normalized feature matrix. You will need to sort eigenvalues by magnitude. How do you do that in Python? Record your explanation as answer #24.

5. Project the data in the feature matrix along the eigenvectors corresponding to the two largest eigenvalues. Plot each observation in this projected space according to quartile of weight at birth. You might need to pick different variables until a rough pattern emerges. Interpret the plot. Record your explanation as answer #25.

Note that you are not expected to simply call a PCA library.

Chapter 7

Introduction to LaTeX

LaTeX is a document preparation system in which the author writes plain text interleaved with markup commands, and a compiler transforms that source into a typeset PDF. This contrasts with word processors such as Microsoft Word or Google Docs, where formatting is applied interactively in a what-you-see-is-what-you-get (WYSIWYG) editor. The key property that makes LaTeX the standard in science and mathematics is its ability to render arbitrarily complex mathematical notation with precision and consistency. Every document produced in this program is a LaTeX document.

7.1 Your First LaTeX Document

A LaTeX source file is a plain text file with the extension `.tex`. Just as `hello.py` was a plain text file that Python interpreted, `hello.tex` is a plain text file that the LaTeX compiler processes into a PDF. The same text editor rules apply (see Section 4.1): use Notepad, TextEdit in plain-text mode, or `gedit`; never a word processor.

Information

Exercise L-1: Your First LaTeX Document Open a plain text editor (see Table 4.1), type the following exactly, and save the file as `hello.tex`:

```
\documentclass{article}
\begin{document}

Hello, world. This is my first \LaTeX{} document.

Here is some mathematics: $e^{i\pi} + 1 = 0$.

\end{document}
```

Each line of the listing serves a specific purpose:

- `\documentclass{article}` declares the document class. For most reports and papers, `article` is the correct choice.
- `\begin{document}` and `\end{document}` delimit the body. Everything outside this pair is the *preamble* (covered in Section 7.3).
- The text between the delimiters is the visible content.
- `$e^{i\pi} + 1 = 0$` is inline math mode: text between dollar signs is typeset as mathematics.

Table 7.1: Compiling a LaTeX document on each platform

Platform	Command
Windows (Command Prompt or Git Bash)	`pdflatex hello.tex`
macOS (Terminal)	`pdflatex hello.tex`
Linux	`pdflatex hello.tex` or `latexmk -pdf hello.tex`

Note that two compilation passes are often required for documents with cross-references or a table of contents; `latexmk` handles this automatically.

Running `pdflatex hello.tex` produces several files. The important one is `hello.pdf`, which can be opened in any PDF viewer. The `.aux`, `.log`, and `.out` files are auxiliary; they can be deleted safely.

7.2 A More Complete Example

Exercise L-1 produced the minimal viable document. This section introduces a more realistic document that demonstrates the features most commonly needed in academic writing: a structured hierarchy of sections, an automatically generated table of contents, in-text citations linked to a bibliography, and the full range of mathematical display modes. Every feature shown here is used in the documents produced by this program.

Information

Exercise L-2: A Structured Document with Math and Bibliography Create two files in the same folder. The first is the main document, `report.tex`. The second is the bibliography database, `sample.bib`. Both files must be in the same directory for the bibliography to resolve.

File 1: `report.tex`

```
\documentclass{article}

\usepackage{amsmath}   % extended math environments
\usepackage{booktabs}  % professional tables
\usepackage{hyperref}  % clickable links and
   cross-references
```

```
6
7  \title{A Short Report on Mathematical Notation}
8  \author{Your Name}
9  \date{\today}
10
11 \begin{document}
12
13 \maketitle
14 \tableofcontents
15 \newpage
16
17 \section{Introduction}
18
19 This report demonstrates document structure and
      mathematical
20 notation in \LaTeX{}. For background on literate
      programming,
21 see Knuth~\cite{knuth1984literate}. The \LaTeX{} system
      itself
22 is described in~\cite{lamport1994latex}.
23
24 \subsection{Motivation}
25
26 Scientific writing requires precise notation. \LaTeX{}
      provides
27 that precision through its mathematical typesetting
      engine.
28
29 \subsubsection{A Note on Style}
30
31 Use display equations for any expression that warrants
      visual
32 emphasis, and inline math for symbols embedded in prose.
33
34 \section{Mathematical Display Modes}
35
36 \subsection{Inline Math}
37
38 Inline math is enclosed in single dollar signs: $f(x) =
      x^2 + 1$.
39 It flows within the surrounding text without a line
      break.
40
41 \subsection{Display Math (unnumbered)}
42
43 The preferred form for unnumbered display equations is
      the
```

```
\texttt{\textbackslash[...\textbackslash]} environment:

\[
    \int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi}
\]

The legacy \texttt{\$\$...\$\$} form produces the same visual
result but is deprecated in \LaTeX{} and should be avoided in
new documents.

\subsection{Numbered Equations}

The \texttt{equation} environment produces a numbered equation
that can be cross-referenced:

\begin{equation}
    \bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i
    \label{eq:mean}
\end{equation}

Equation~\ref{eq:mean} defines the sample mean. An approximation
for the standard deviation is given in~\cite{gurland1971simple}.

\subsection{Multi-line Aligned Equations}

The \texttt{align} environment aligns multiple equations at a
designated column, marked with \texttt{\&}:

\begin{align}
    \mu    &= \mathbb{E}[X]            \label{eq:mu} \\
    \sigma^2 &= \mathbb{E}[(X-\mu)^2] \label{eq:var}
\end{align}

Equations~\ref{eq:mu} and~\ref{eq:var} define the mean and
variance of a random variable $X$.

\section{A Simple Table}

\begin{table}[htbp]
```

```
\centering
\caption{Comparison of equation display modes}
\label{tab:modes}
\begin{tabular}{lll}
\toprule
Mode & Syntax & Use case \\
\midrule
Inline     & \texttt{\$...\$}                    &
   Symbols in prose \\
Unnumbered & \texttt{\textbackslash[...\textbackslash]}
    & Emphasis, no reference needed \\
Numbered   & \texttt{equation}                   &
   Cross-referenceable \\
Aligned    & \texttt{align}                      &
   Multi-line derivations \\
\bottomrule
\end{tabular}
\end{table}

\bibliographystyle{plain}
\bibliography{sample}

\end{document}
```

File 2: `sample.bib`

```
@article{knuth1984literate,
  author  = {Knuth, Donald E.},
  title   = {Literate Programming},
  journal = {The Computer Journal},
  year    = {1984},
  volume  = {27},
  number  = {2},
  pages   = {97--111}
}

@book{lamport1994latex,
  author    = {Lamport, Leslie},
  title     = {\LaTeX: A Document Preparation System},
  edition   = {2nd},
  publisher = {Addison-Wesley},
  year      = {1994}
}

@article{gurland1971simple,
  author  = {Gurland, John and Tripathi, Ram C.},
  title   = {A Simple Approximation for Unbiased
```

```
    Estimation
22              of the Standard Deviation},
23  journal = {The American Statistician},
24  year    = {1971},
25  volume  = {25},
26  number  = {4},
27  pages   = {30--32}
28 }
```

Warning

Compiling with a bibliography requires four steps A single `pdflatex` pass is not sufficient when the document contains `\cite{}` commands. The full sequence is:

```
pdflatex report.tex
bibtex report
pdflatex report.tex
pdflatex report.tex
```

The first pass creates the `.aux` file listing all citation keys. `bibtex` reads the `.aux` file and the `.bib` database to produce a `.bbl` file containing the formatted bibliography. The second and third `pdflatex` passes incorporate the bibliography and resolve all cross-references. Running `latexmk -pdf report.tex` performs all necessary passes automatically.

Table 7.2: Equation display modes in LaTeX

Mode	Syntax	Output style	Notes
Inline	`$...$`	Within text, compact	Use for symbols and short expressions in prose
Display (legacy)	`$$...$$`	Centered, full size	Deprecated; avoid in new documents
Display (preferred)	`\[...\]`	Centered, full size	Use for unnumbered display equations
Numbered	`\begin{equation} ...\end{equation}`	Centered with number	Cross-referenceable via `\label`/`\ref`
Multi-line aligned	`\begin{align} ...\end{align}`	Multiple lines, aligned at `&`	Each line can carry its own `\label`

7.3 Document Structure

Every LaTeX source file has two parts: the *preamble* (everything before `\begin{document}`) and the *body* (everything between `\begin{document}` and `\end{document}`). The preamble declares the document class, loads packages with `\usepackage{...}`, and sets metadata. The body contains the visible content, organized with sectioning commands.

```
\documentclass{article}

% --- Preamble ---
\usepackage{amsmath}    % extended math
\usepackage{graphicx}   % figures
\usepackage{booktabs}   % professional tables
\usepackage{hyperref}   % clickable links

\title{My First Report}
\author{Alice Smith}
\date{\today}

% --- Body ---
\begin{document}

\maketitle

\section{Introduction}
This is the introduction.

\subsection{Background}
Some background material.

\subsubsection{Details}
More specific details.

\section{Methods}
Description of methods.

\end{document}
```

Listing 7.1: Minimal compilable document with preamble and sectioning

The `\usepackage{...}` command loads extension packages. The packages most relevant to this program are `amsmath` (extended math), `graphicx` (figures), `booktabs` (tables), and `hyperref` (clickable links).

7.4 Mathematics in LaTeX: A Cheat Sheet

LaTeX distinguishes two math modes. Inline mode (`$...$`) typesets mathematics within a line of text. Display mode (`\[...\]` or the `equation` environment) centers

mathematics on its own line and assigns an equation number when using `equation`. Use display mode for any expression that warrants its own visual emphasis, and inline mode for symbols embedded in prose.

7.4.1 Common Mathematical Notation

Table 7.3: Common LaTeX mathematical notation

Category	LaTeX source	Rendered	Notes
Inline and display modes			
Inline math	`$x + y = z$`	$x + y = z$	Within a sentence
Display math	`\[ x + y = z \]`	(centered block)	No number
Numbered equation	`\begin{equation}...\end{equation}`	(with number)	Cross-referenceable
Basic arithmetic			
Addition	`a + b`	$a + b$	
Subtraction	`a - b`	$a - b$	
Multiplication (dot)	`a \cdot b`	$a \cdot b$	Preferred in math
Multiplication (cross)	`a \times b`	$a \times b$	Vectors, cross product
Division	`\frac{a}{b}`	$\frac{a}{b}$	Always use `\frac` in display
Superscript	`x^{2}`	x^2	Braces required for ¿1 char
Subscript	`x_{i}`	x_i	Braces required for ¿1 char
Both	`x_{i}^{2}`	x_i^2	
Common functions			
Square root	`\sqrt{x}`	$\sqrt{x}$	
nth root	`\sqrt[n]{x}`	$\sqrt[n]{x}$	
Exponential	`e^{x}` or `\exp(x)`	e^x	`\exp` preferred in display
Natural log	`\ln x`	$\ln x$	
Log base b	`\log_{b} x`	$\log_b x$	
Sine	`\sin x`	$\sin x$	Use `\sin`, not `sin`
Cosine	`\cos x`	$\cos x$	
Absolute value	`|x|` or `\lvert x \rvert`	$\|x\|$	`\lvert` scales with content
Floor	`\lfloor x \rfloor`	$\lfloor x \rfloor$	
Ceiling	`\lceil x \rceil`	$\lceil x \rceil$	

Continued on next page

Category	LaTeX source	Rendered	Notes
Summation and integration			
Sum	`\sum_{i=1}^{n} x_i`	$\sum_{i=1}^{n} x_i$	Limits above/below in display
Product	`\prod_{i=1}^{n} x_i`	$\prod_{i=1}^{n} x_i$	
Integral	`\int_{a}^{b} f(x)\,dx`	$\int_a^b f(x)\,dx$	`\,` adds thin space before dx
Limit	`\lim_{x \to 0} f(x)`	$\lim_{x \to 0} f(x)$	
Greek letters (common in biostatistics and ML)			
alpha	`\alpha`	α	Significance level, learning rate
beta	`\beta`	β	Regression coefficients
gamma	`\gamma`, `\Gamma`	γ, Γ	Gamma distribution
delta	`\delta`, `\Delta`	δ, Δ	Change, Dirac delta
epsilon	`\epsilon`, `\varepsilon`	ϵ, ε	Small quantity
theta	`\theta`, `\Theta`	θ, Θ	Parameters, angle
lambda	`\lambda`, `\Lambda`	λ, Λ	Poisson rate, eigenvalue
mu	`\mu`	μ	Mean
nu	`\nu`	ν	Degrees of freedom
pi	`\pi`, `\Pi`	π, Π	3.14..., product
rho	`\rho`	ρ	Correlation
sigma	`\sigma`, `\Sigma`	σ, Σ	Std dev, summation
tau	`\tau`	τ	Time constant
phi	`\phi`, `\Phi`	ϕ, Φ	Normal CDF, angle
chi	`\chi`	χ	Chi-squared test
omega	`\omega`, `\Omega`	ω, Ω	Frequency, sample space
Relations and logic			
Not equal	`\neq`	$\neq$	
Less/greater or equal	`\leq`, `\geq`	$\leq$, $\geq$	
Approximately	`\approx`	$\approx$	
Proportional to	`\propto`	$\propto$	
Distributed as	`\sim`	$\sim$	
For all	`\forall`	$\forall$	
There exists	`\exists`	$\exists$	

Continued on next page

Category	LaTeX source	Rendered	Notes
Implies	`\Rightarrow`	$\Rightarrow$	
If and only if	`\Leftrightarrow`	$\Leftrightarrow$	
Element of	`\in`	$\in$	
Subset	`\subset`, `\subseteq`	$\subset$, $\subseteq$	
Union, intersection	`\cup`, `\cap`	$\cup$, $\cap$	
Number sets			
Real numbers	`\mathbb{R}`	$\mathbb{R}$	Requires `amsfonts` or `amssymb`
Natural numbers	`\mathbb{N}`	$\mathbb{N}$	
Integers	`\mathbb{Z}`	$\mathbb{Z}$	
Rationals	`\mathbb{Q}`	$\mathbb{Q}$	
Complex	`\mathbb{C}`	$\mathbb{C}$	
Vectors and matrices			
Bold vector	`\mathbf{x}`	$\mathbf{x}$	Preferred in statistics
Vector with arrow	`\vec{x}`	$\vec{x}$	Physics convention
Transpose	`A^{\top}` or `A^T`	$A^\top$	
2×2 matrix	`\begin{bmatrix} a & b \\ c & d \end{bmatrix}`	$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$	Requires `amsmath`
Norm	`|x|`	$\|x\|$	
Inner product	`\langle x, y \rangle`	$\langle x, y \rangle$	

7.5 Figures

Figures are included using the `figure` float environment and the `\includegraphics` command from the `graphicx` package. The `pdflatex` compiler accepts PDF, PNG, and JPG image formats.

```
\begin{figure}[htbp]
    \centering
    \includegraphics[width=0.7\textwidth]{images/my-plot.png}
    \caption{Description of the figure.}
    \label{fig:my-plot}
\end{figure}
```

Listing 7.2: Including a figure in LaTeX

The `[htbp]` option suggests placement preferences to LaTeX: here, top, bottom, or a dedicated page. The `\caption` and `\label` commands provide a numbered caption and a cross-reference target, respectively.

7.6 Tables

Tables are built with the `table` float and the `tabular` environment. The `booktabs` package provides clean horizontal rules. Vertical rules (`|`) are omitted by convention in professional documents; `booktabs` horizontal rules are sufficient.

```
\begin{table}[htbp]
\centering
\caption{Sample measurements}
\label{tab:sample}
\begin{tabular}{lrr}
\toprule
Subject & Height (cm) & Weight (kg) \\
\midrule
Alice   & 165         & 60          \\
Bob     & 178         & 75          \\
Carol   & 172         & 68          \\
\bottomrule
\end{tabular}
\end{table}
```

Listing 7.3: A minimal table with booktabs

7.7 Bibliography

LaTeX manages bibliographies via an external file (`.bib`) and a style command. The `\cite{key}` command inserts a citation at any point in the text; the `\bibliography{filename}` and `\bibliographystyle{plain}` commands generate the reference list.

For more sophisticated bibliography management, the `biblatex` package with the `biber` backend is preferred in modern documents. This repository's shared preamble has bibliography support commented out; students writing independent reports should uncomment and configure it.

7.8 LaTeX Workshop in Visual Studio Code

The LaTeX Workshop extension for VSC (referenced in Section 2.4) provides a complete integrated workflow: syntax highlighting, live PDF preview, one-click build, and error navigation. Once installed, opening any `.tex` file activates the extension automatically. The build shortcut is `Ctrl+Alt+B` (Windows/Linux) or `Cmd+Option+B` (macOS), which runs `pdflatex` and displays the result in the built-in PDF viewer. Students do not need to use the command line for compilation after this is set up; the command-line approach in Section 7.1 is presented first so the underlying process is understood.

7.9 Summary

This chapter introduced LaTeX, the document preparation system used throughout this program and widely adopted as the standard in science and mathematics:

- **What LaTeX is.** A plain-text markup language compiled into typeset PDFs, in contrast to WYSIWYG word processors.
- **The source-to-PDF workflow.** Writing a `.tex` file in a text editor, compiling with `pdflatex` or `latexmk`, and viewing the resulting PDF.
- **Document structure.** The preamble (document class, packages, metadata) and the body (content organized with `\section`, `\subsection`, and `\subsubsection`).
- **Inline and display math.** Dollar signs for inline mathematics and `\[...\]` or the `equation` environment for display mathematics.
- **Equation modes.** Five modes exist: inline (`$...$`), legacy display (`$$...$$`, deprecated), preferred unnumbered display (`\[...\]`), numbered (`equation`), and multi-line aligned (`align`). Use `\[...\]` for unnumbered and `equation` for cross-referenceable display equations.
- **The math cheat sheet.** A comprehensive reference table (Table 7.3) covering arithmetic, functions, summation, integration, Greek letters, relations, number sets, and vectors/matrices.
- **Figures and tables.** Float environments with `\includegraphics`, `tabular`, and `booktabs` for professional formatting.
- **Bibliography workflow.** A `.bib` file stores reference entries. The `\cite{key}` command inserts a citation; `\bibliography{filename}` and `\bibliographystyle{plain}` generate the reference list. Documents with citations require a four-step compilation sequence: two `pdflatex` passes enclosing one `bibtex` pass, followed by a final `pdflatex`. `latexmk` handles this automatically.
- **LaTeX Workshop.** The VSC extension that provides syntax highlighting, live preview, and one-click build, eliminating the need for command-line compilation in daily use.

Chapter 8

Version Control with Git and GitHub

8.1 Configuring SSH Authentication for GitHub on Linux

GitHub no longer supports password authentication for Git operations over HTTPS. Instead, authentication must be performed using either a Personal Access Token or an SSH key. The recommended method for Linux environments is SSH authentication.

The following procedure generates an SSH key and configures Git to authenticate with GitHub.

8.1.1 Generate an SSH Key

Create a new SSH key using the `ed25519` algorithm:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

When prompted for the file location, press `Enter` to accept the default:

```
~/.ssh/id_ed25519
```

Optionally provide a passphrase for additional security.

8.1.2 Start the SSH Agent

Start the SSH agent and add the newly created key:

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

8.1.3 Add the Public Key to GitHub

Display the public key:

```
cat ~/.ssh/id_ed25519.pub
```

Copy the entire output and add it to GitHub:

1. Navigate to `GitHub → Settings → SSH and GPG keys`
2. Click `New SSH key`
3. Paste the copied key and save

8.1.4 Verify the Connection

Test the authentication:

```
ssh -T git@github.com
```

A successful configuration will display a message similar to:

```
Hi <username>! You've successfully authenticated.
```

8.1.5 Clone a Repository Using SSH

Repositories must be cloned using the SSH URL:

```
git clone git@github.com:USER/REPOSITORY.git
```

Using SSH avoids credential prompts and enables secure automated Git operations.

8.2 Configuring SSH Authentication for GitHub on Windows

Git supports two protocols for communicating with remote repositories: HTTPS and SSH. While HTTPS is simpler to set up initially, SSH authentication offers a more seamless workflow for frequent contributors because it eliminates repeated credential prompts. This section walks through the complete process of configuring SSH-based authentication for GitHub on a Windows machine using Git Bash.

8.2.1 Opening Git Bash

Git for Windows ships with its own bundled OpenSSH client and a Unix-like shell called Git Bash. All SSH commands in this section must be run inside Git Bash, not in Command Prompt or PowerShell, because the bundled `ssh`, `ssh-keygen`, and `ssh-agent` executables are located in the Git for Windows installation directory and are not automatically available in other terminals.

To open Git Bash, right-click on the Desktop or any folder in File Explorer and select **Git Bash Here** from the context menu. Alternatively, search for **Git Bash** in the Windows Start menu and launch it from there.

8.2.2 Generating an SSH Key

SSH authentication relies on a public/private key pair. The private key remains on your machine; the public key is uploaded to GitHub. Generate a new key pair using the `ed25519` algorithm, which is modern, compact, and recommended over the older RSA default:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Listing 8.1: Generate an ed25519 SSH key pair

Replace `your_email@example.com` with the address associated with your GitHub account. When prompted for a file location, press **Enter** to accept the default path:

```
C:\Users\YourName\.ssh\id_ed25519
```

Listing 8.2: Default key location on Windows

You will then be asked to enter an optional passphrase. A passphrase adds a second layer of protection in case your private key file is ever exposed; however, it must be entered each time the key is used (unless you configure the SSH agent to cache it). Press **Enter** twice to skip the passphrase if you prefer not to set one.

8.2.3 Starting the SSH Agent

The SSH agent is a background process that holds decrypted private keys in memory, so you do not have to re-enter a passphrase for every Git operation. On Linux and macOS the agent is typically started automatically by the desktop session, but on Windows the `ssh-agent` bundled with Git Bash must be started manually in each new terminal session.

Start the agent and add your private key with the following two commands:

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

Listing 8.3: Start the SSH agent and add the private key

The first command launches the agent and sets the necessary environment variables in the current shell. The second command registers your private key with the running agent. If you set a passphrase during key generation, you will be prompted to enter it now.

Tip

Automating Agent Startup The commands above must be repeated every time you open a new Git Bash session. To avoid this, you can add the following lines to your `~/.bashrc` file so that the agent starts automatically:

```
eval "$(ssh-agent -s)" > /dev/null 2>&1
ssh-add ~/.ssh/id_ed25519 2>/dev/null
```

Open or create `~/.bashrc` in a text editor, paste the two lines at the end of

the file, and save. The redirects suppress startup messages so they do not clutter your terminal. The next time you open Git Bash the agent will start and load your key without any manual steps.

8.2.4 Adding the Public Key to GitHub

GitHub needs your *public* key to verify your identity. Display its contents with:

```
cat ~/.ssh/id_ed25519.pub
```

Listing 8.4: Display the public key

The output will be a single long line beginning with `ssh-ed25519`. Select and copy the entire line, including the email address at the end.

Next, navigate to GitHub in your browser and follow these steps:

1. Click your profile photo in the upper-right corner and select **Settings**.
2. In the left sidebar, click **SSH and GPG keys**.
3. Click **New SSH key**.
4. Give the key a descriptive title (for example, *Windows laptop*).
5. Paste the copied public key into the **Key** field.
6. Click **Add SSH key** and confirm with your GitHub password if prompted.

8.2.5 Verifying the Connection

Before using SSH for any repository operations, confirm that GitHub can authenticate your key:

```
ssh -T git@github.com
```

Listing 8.5: Test the SSH connection to GitHub

On the first connection, SSH will display a fingerprint and ask whether you trust the host. Type `yes` and press **Enter**. If authentication succeeds, you will see a message similar to:

```
Hi username! You've successfully authenticated, but GitHub
    does
not provide shell access.
```

Listing 8.6: Expected response from GitHub

If instead you receive a `Permission denied` error, verify that the agent is running (`ssh-add -l` should list your key) and that the correct public key has been added to your GitHub account.

8.2.6 Cloning a Repository Using SSH

Once authentication is confirmed, clone repositories using the SSH remote URL:

```
git clone git@github.com:USER/REPOSITORY.git
```

Listing 8.7: Clone a repository over SSH

Replace `USER` with the repository owner's GitHub username and `REPOSITORY` with the repository name.

Warning

Use the SSH Remote URL, Not HTTPS The SSH remote URL begins with `git@github.com:` and ends with `.git`. It is *not* the same as the HTTPS URL, which begins with `https://github.com/`. If you clone or push using the HTTPS form after configuring SSH authentication, Git will still prompt you for a GitHub username and password (or personal access token). Always verify the remote URL with `git remote -v` and update it if necessary:

```
git remote set-url origin
    ↪ git@github.com:USER/REPOSITORY.git
```

Table 8.1: SSH setup comparison: Linux/macOS vs. Windows (Git Bash).

Step	Linux / macOS	Windows (Git Bash)
Open terminal	Open the system terminal application (Terminal, iTerm2, etc.)	Right-click Desktop or folder and select *Git Bash Here*, or search Git Bash in the Start menu
Generate key	`ssh-keygen -t ed25519 -C "email"` (stored in `~/.ssh/`)	Same command in Git Bash; default path is `C:\Users\Name\.ssh\`
Start agent	Agent is typically started automatically by the desktop session; run `eval "$(ssh-agent -s)"` if needed	Must be started manually each session: `eval "$(ssh-agent -s)"`
Add key to agent	`ssh-add ~/.ssh/id_ed25519`	Same command in Git Bash
Display public key	`cat ~/.ssh/id_ed25519.pub`	Same command in Git Bash
Verify connection	`ssh -T git@github.com`	Same command in Git Bash
Clone with SSH	`git clone git@github.com:USER/REPO.git`	Same command in Git Bash

8.2.7 Pushing to Multiple Remote Repositories

A single local repository can maintain simultaneous connections to any number of remote repositories. This is useful when a project must be mirrored across an organisation account and a personal account (for example, keeping the ELEVATE Academy workshop materials synchronised between the `ELEVATE Academy` organisation and the personal `YourVeryOwnGitHubAccount` GitHub account).

8.2.7.1 Concepts

Git identifies remote repositories by short *names*. The name `origin` is the conventional default assigned when a repository is first cloned. Additional remotes can be registered under any name. Each remote entry stores a fetch URL and a push URL, both of which can be set independently.

Information

A remote name is purely local: it exists only in the `.git/config` file on your machine and is never pushed to any server. Two developers cloning the same repository can give the same remote entirely different names without consequence.

8.2.7.2 Listing Registered Remotes

Before adding a remote, confirm what is already registered:

```
git remote -v
```

Listing 8.8: List all remotes and their URLs

Each remote appears twice, once for fetch and once for push. A freshly cloned repository typically shows only `origin`:

```
origin  https://github.com/biomathematicus/ELEVATE.git (fetch)
origin  https://github.com/biomathematicus/ELEVATE.git (push)
```

8.2.7.3 Adding a Second Remote

Register an additional remote with `git remote add`:

```
git remote add YourVeryOwnGitHubAccount \
    https://github.com/biomathematicus/ELEVATE.git
```

Listing 8.9: Register a personal mirror remote (Linux / WSL)

```
git remote add YourVeryOwnGitHubAccount `
    https://github.com/biomathematicus/ELEVATE.git
```

Listing 8.10: Register a personal mirror remote (PowerShell)

Verify the result:

```
git remote -v
```

The output should now show four lines:

```
YourVeryOwnGitHubAccount
    https://github.com/YourVeryOwnGitHubAccount/ELEVATE
    Academy.git (fetch)
YourVeryOwnGitHubAccount
    https://github.com/YourVeryOwnGitHubAccount/ELEVATE
    Academy.git (push)
origin            https://github.com/biomathematicus/ELEVATE.git
    (fetch)
origin            https://github.com/biomathematicus/ELEVATE.git
    (push)
```

8.2.7.4 Pushing to Each Remote

With two remotes registered, every push must be issued explicitly to each destination. Git does not push to multiple remotes automatically unless a push target is configured:

```
git push origin HEAD
git push YourVeryOwnGitHubAccount HEAD
```

Listing 8.11: Push to both remotes explicitly

`HEAD` resolves to the current branch, which is the safest form to use in scripts because it does not depend on knowing the branch name in advance.

8.2.7.5 Checking Whether a Remote Exists Before Pushing

In automated scripts it is good practice to verify that a remote is registered before attempting to push to it, so a missing remote produces a clear diagnostic rather than a fatal error.

```
if git remote get-url YourVeryOwnGitHubAccount > /dev/null
    ↪ 2>&1; then
    git push YourVeryOwnGitHubAccount HEAD
else
    echo "Remote 'YourVeryOwnGitHubAccount' not configured."
    echo "Register it with:"
    echo "  git remote add YourVeryOwnGitHubAccount \
https://github.com/YourVeryOwnGitHubAccount/ELEVATE
    Academy.git"
fi
```

Listing 8.12: Guard a push with a remote existence check (bash)

```
git remote get-url YourVeryOwnGitHubAccount >nul 2>&1
if errorlevel 1 (
    echo Remote 'YourVeryOwnGitHubAccount' not configured.
```

```
    echo Register it with:
    echo   git remote add YourVeryOwnGitHubAccount ^
        https://github.com/YourVeryOwnGitHubAccount/ELEVATE
           Academy.git
) else (
    git push YourVeryOwnGitHubAccount HEAD
)
```

Listing 8.13: Guard a push with a remote existence check (PowerShell / batch)

8.2.7.6 Managing Remotes

Table 8.2 summarises the full set of remote management commands.

Table 8.2: Git remote management commands

Command	Effect
`git remote -v`	List all remotes with their fetch and push URLs.
`git remote add <n> <url>`	Register a new remote under the given name.
`git remote remove <n>`	Delete a remote entry from the local configuration.
`git remote rename <old> <new>`	Rename a remote without changing its URL.
`git remote set-url <n> <url>`	Update the URL of an existing remote.
`git remote get-url <n>`	Print the URL of a named remote; exits with an error if the remote does not exist (useful in scripts).
`git remote show <n>`	Display detailed information about a remote, including tracked branches and push configuration.

8.2.7.7 Alternative: a Single Remote with Multiple Push URLs

Git also supports configuring one remote name to push to two URLs simultaneously, so that a single `git push origin HEAD` reaches both destinations. This is convenient but reduces visibility; a failure on the second URL can be easy to miss in the output.

```
# The first push URL is already set from the clone.
# Add the personal repo as an additional push target:
git remote set-url --add --push origin \
    https://github.com/biomathematicus/ELEVATE.git
git remote set-url --add --push origin \
    https://github.com/YourVeryOwnGitHubAccount/ELEVATE
       Academy.git
```

Listing 8.14: Add a second push URL to the origin remote

Warning

When using `set-url --add --push`, note that the *first* call replaces the default push URL rather than adding to it. Both URLs must be specified explicitly as shown above. Verify the result with `git remote -v` before relying on this configuration in automated scripts.

For the Alice project, the explicit two-remote approach (separate `git push` calls per remote) is preferred because `gh.bat` checks `errorlevel` after each push and can report failures independently. A combined push URL would surface only the second failure.

8.3 Summary

This chapter introduced version control with Git and GitHub, focusing on the practical steps required to authenticate, clone, and synchronise repositories.

- **SSH key-based authentication.** GitHub no longer supports password authentication for Git operations. This chapter covered the complete SSH setup procedure on both Linux and Windows (using Git Bash), including key generation with the `ed25519` algorithm, adding the public key to GitHub, and verifying the connection.
- **Cloning repositories via SSH.** Once authentication is configured, repositories are cloned using the `git@github.com:` URL form, which avoids credential prompts on every push and pull.
- **Multiple remote repositories.** A single local repository can push to more than one remote. This chapter demonstrated how to register a second remote and push to both, keeping organisation and personal copies in sync.
- **Automated staging, committing, and pushing with `gh.bat`.** The `gh.bat` script automates the add-commit-push cycle, including version stamping and pushing to both remotes, reducing repetitive manual steps to a single command.

Chapter 9

Local LLM Infrastructure with Ollama

9.1 Introduction

For students: this chapter has two parts relevant to you. Section 9.7 explains how to configure the API keys required to run the agent programs in `resources/unit_7/FOO/`. The Ollama CLI reference in Section 9.4 covers the commands you will use during the LLM unit exercises. You do not need to read the remainder of this chapter to complete the data challenge.

For instructors and program staff: this chapter documents the full configuration of the two-machine LLM serving infrastructure used by the ELEVATE Academy program, covering the Dell Precision 7770 application host and the NVIDIA Spark inference server. It includes Ollama installation, `systemd` service configuration, network access control, memory budget management, single-machine and two-machine deployment modes, connectivity testing, and capacity planning.

9.2 Hardware Inventory

The two machines participating in this setup have fundamentally different memory architectures, which governs model selection and agent capacity.

9.2.1 Dell Precision 7770 (Application Host)

- **CPU RAM:** 128 GB (112 GB available after OS overhead)
- **GPU:** NVIDIA RTX A5500 Laptop GPU, 16 GB VRAM (discrete)
- **Architecture:** x86_64
- **OS:** Windows 11 with WSL2 Ubuntu 24.04

- **Role:** Runs a FastAPI application, PostgreSQL 16, and the agent orchestration layer. Hosts a local Ollama instance for single-machine development mode.

On the Dell, the RTX A5500 VRAM (16 GB) is *separate* from system RAM. Ollama loads model weights into VRAM first and spills to system RAM only when the model exceeds VRAM capacity. Inference degrades significantly once spilling begins: tokens per second can drop by an order of magnitude for spilled portions of the model.

9.2.2 NVIDIA Spark (LLM Server)

- **Unified memory:** 128 GB physical; approximately 120 GB visible to Ollama (110 GB conservatively available)
- **GPU:** NVIDIA GB10 SoC
- **Architecture:** ARM64 (AArch64)
- **OS:** Ubuntu 24.04 (native, no virtualisation)
- **Role:** Dedicated LLM inference server. CPU and GPU share the same memory pool, so the entire 110 GB is available for model weights and KV cache simultaneously.

Warning

The GB10 uses a *unified memory* architecture. Unlike a discrete GPU, there is no separate VRAM budget to track. The 110 GB pool covers weights, KV cache, and OS overhead together. This makes the Spark well suited to serving multiple large models simultaneously (provided context windows are bounded), but it also means that vLLM's PagedAttention, which is optimised for discrete high-VRAM GPUs, provides fewer benefits here than Ollama's native unified-memory handling.

9.3 Network Topology

Both machines reside on the UTSA `utsarr.net` subnet. At the time of configuration they held the following addresses:

Machine	Interface	IP Address	Subnet
Spark	`enP7s7`	`10.2.14.24`	`/23`
Dell	Ethernet 8	`10.2.14.143`	`/23`
Spark	`enP7s7` MAC	`4c:bb:47:2c:74:eb`	
Dell	Ethernet 8 MAC	`ac:91:a1:7d:06:ff`	

9.3.1 Switch vs. Router

A Netgear ProSAFE GS105 unmanaged 5-port switch connects the two machines. An *unmanaged switch* operates at OSI Layer 2: it forwards Ethernet frames between ports based on MAC addresses but has no concept of IP subnets, routing, or traffic restriction. A *router* operates at Layer 3 and is required to separate subnets or enforce IP-level traffic policies.

Because both machines are on the same `/23` subnet managed by the university network, ARP entries are not visible between hosts; traffic crosses at least one router hop, and ARP does not traverse routers. This is expected behaviour on a routed network and should not be interpreted as a connectivity error.

9.3.2 MAC Address Filtering

MAC-based firewall rules (via `iptables`) are only meaningful on the same Layer 2 segment. They can be spoofed trivially and should be treated as an informational layer rather than a security boundary. IP firewall rules (`ufw`) are the appropriate primary control for restricting Ollama access to known client addresses.

```
# On the Spark: allow only the Dell, deny all other sources
sudo ufw allow from 10.2.14.143 to any port 11434
sudo ufw deny 11434
```

Listing 9.1: Restrict Ollama to the Dell's IP only

9.3.3 Identifying Network Interfaces

Use the following commands to enumerate interfaces and confirm addresses before configuring firewall rules.

```
ip a
# Look for interfaces with state UP and a valid inet
   ↪ address.
# The link/ether line on each interface gives the MAC
   ↪ address.
```

Listing 9.2: Linux: list interfaces and addresses

```
ipconfig /all
# Look for active Ethernet adapters with an IPv4 Address.
# The Physical Address field gives the MAC (dash-separated
   ↪ format).
```

Listing 9.3: Windows: list interfaces and addresses

Information

MAC address formats. Linux tools (`ip a`, `arp`) report MACs in lowercase colon format: `ac:91:a1:7d:06:ff`. Windows `ipconfig /all` reports them in uppercase dash format: `AC-91-A1-7D-06-FF`. Both represent the same

address. Configuration files and scripts should normalise to a single format before comparison.

9.4 Installing Ollama

9.4.1 Linux (Spark)

The official installer script detects the architecture automatically and selects the appropriate binary (ARM64 in this case).

```
curl -fsSL https://ollama.com/install.sh | sh
```

Listing 9.4: Install Ollama on Ubuntu

The installer creates the `ollama` user and group, adds the current user to the `ollama` group, installs the binary to `/usr/local/bin/ollama`, and registers a `systemd` service. Confirm the installation:

```
ollama --version
systemctl status ollama
```

9.4.2 Windows (Dell)

Download the Windows installer from `https://ollama.com` and run it. Ollama registers itself in the system tray and starts automatically. The API listens on `127.0.0.1:11434` by default.

9.4.3 Conda Environments

The Spark ships with Miniconda, which activates a `(base)` environment by default. Ollama does not require Python and should be installed at the system level, not inside a Conda environment. Disable automatic Conda activation to avoid interference with system tooling:

```
conda config --set auto_activate_base false
```

For all Python work in this stack, use plain `venv` environments, which translate directly to `systemd ExecStart` paths and maintain production parity without Conda's dependency overhead.

9.5 Configuring Ollama for Network Access

By default Ollama binds only to `localhost`. For the two-machine setup the Spark must expose its API to the network. Configure this through a `systemd` override so the setting survives service restarts and reboots.

```
sudo systemctl edit ollama
```

Listing 9.5: Create a systemd override on the Spark

This opens `/etc/systemd/system/ollama.service.d/override.conf`. Add the following:

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0:11434"
Environment="OLLAMA_MAX_LOADED_MODELS=4"
Environment="OLLAMA_NUM_PARALLEL=4"
Environment="OLLAMA_KEEP_ALIVE=24h"
Environment="OLLAMA_NUM_CTX=8192"
```

Listing 9.6: Spark Ollama systemd override

Apply the changes:

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
systemctl status ollama
```

The environment variables are explained in Table 9.1.

Table 9.1: Key Ollama environment variables

Variable	Effect
`OLLAMA_HOST`	Bind address and port. Use `0.0.0.0:11434` to accept connections from any network interface.
`OLLAMA_MAX_LOADED_MODELS`	Maximum number of models resident in memory simultaneously. Prevents thrashing when Max dispatches concurrent agent calls.
`OLLAMA_NUM_PARALLEL`	Maximum concurrent inference requests per model.
`OLLAMA_KEEP_ALIVE`	Duration a model remains loaded after its last request. The default is 5 minutes; `24h` keeps models warm across a workday.
`OLLAMA_NUM_CTX`	Default context window in tokens. **Critical on the Spark**: without this cap, Ollama defaults to 262,144 tokens, which allocates a KV cache large enough to exceed physical memory on most models. Set to `8192` as a safe default.

9.6 Memory Budget and Context Window Management

9.6.1 The Governing Formula

Total memory consumed per agent has two independent components:

$$M_{\text{agent}} = M_{\text{weights}} + M_{\text{KV}} \tag{9.1}$$

where the weight footprint under Q4 quantisation is approximated by:

$$M_{\text{weights}} \approx 0.5\,\text{GB} \times B_{\text{params}} \tag{9.2}$$

and the KV cache footprint is:

$$M_{\text{KV}} \approx \frac{n_{\text{layers}} \times n_{\text{kv\_heads}} \times d_{\text{head}} \times C \times 2 \times 2}{10^9}\,\text{GB} \tag{9.3}$$

where C is the context length in tokens and the factor of 2 appears twice for the key/value pair and the 2-byte (BF16) dtype.

Warning

The context window dominates memory for large models. A `llama3.3:70b` model requires approximately 40 GB for weights at Q4. With Ollama's default context of 262,144 tokens, the KV cache adds another 134 GB, pushing total allocation to 174 GB, exceeding the Spark's 120 GB and causing the load to fail. With a context of 8,192 tokens, the KV cache drops to 2.5 GB, giving a total of 42.5 GB which fits comfortably.

9.6.2 The Modelfile Pattern for Large Models

Ollama does not support runtime context overrides via the `--num-ctx` flag. The correct mechanism is a *Modelfile*, which creates a named derivative of the base model with the parameter baked in.

9.6.2.1 Linux (Spark and WSL)

```
cat > /tmp/Modelfile.llama70b << 'EOF'
FROM llama3.3:70b
PARAMETER num_ctx 8192
EOF
ollama create llama3.3:70b-ctx8k -f /tmp/Modelfile.llama70b
ollama run llama3.3:70b-ctx8k "Reply with one word: ready"
```

Listing 9.7: Create a context-bounded model variant on Linux

9.6.2.2 Windows (PowerShell)

PowerShell uses a here-string syntax for multi-line literals. The closing `"@` must appear at the *absolute start of a line* with no leading spaces or tabs; this is a hard requirement of the PowerShell parser.

```
@"
FROM llama3.3:70b
PARAMETER num_ctx 8192
```

```
"@ | Out-File -FilePath "$env:TEMP\Modelfile.llama70b"
   ↪ -Encoding utf8
ollama create llama3.3:70b-ctx8k -f
   ↪ "$env:TEMP\Modelfile.llama70b"
```

Listing 9.8: Create a context-bounded model variant on Windows

Apply the same pattern to every model above 20 B parameters. The naming convention `<model>-ctx8k` makes the operational constraint explicit at the call site.

9.6.3 Hardware Agent Capacity

Given the memory budget formula, the maximum number of agents that can run simultaneously on a machine is:

$$N_{\text{hw}} = \left\lfloor \frac{M_{\text{available}}}{M_{\text{agent}}} \right\rfloor \tag{9.4}$$

Table 9.2 gives the confirmed model inventory with measured status on each platform and approximate memory footprint at 8,192-token context.

Table 9.2: Confirmed model inventory and memory footprint at 8K context

Model	Ollama name	Params (B)	M_w (GB)	M_{KV} (GB)	M_{total} (GB)	Status
nemotron-3-super:120b	`...-ctx8k`	120 (MoE)	65	4.0	69.0	✓ both
llama3.3:70b	`...-ctx8k`	70	40	2.5	42.5	✓ both
qwen3:32b	`...-ctx8k`	32	18	1.3	19.3	✓ both
mistral-small:22b	`mistral-small:22b`	22	13	1.0	14.0	✓ both
phi4:14b	`phi4:14b`	14	8	0.8	8.8	✓ both
qwen3:14b	`qwen3:14b`	14	8	0.8	8.8	✓ both
mistral-nemo:12b	`mistral-nemo:12b`	12	7	0.7	7.7	✓ both
mistral:7b	`mistral:7b`	7	4	0.5	4.5	✓ both
deepseek-r1:8b	`deepseek-r1:8b`	8	5	0.5	5.5	✓ both
gemma3:4b	`gemma3:4b`	4	2.5	0.3	2.8	✓ both
phi4-mini:3.8b	`phi4-mini:3.8b`	3.8	2.5	0.3	2.8	✓ both
qwen3:4b	`qwen3:4b`	4	2.5	0.3	2.8	✓ Dell; † Spark
gemma3:1b	`gemma3:1b`	1	0.8	0.1	0.9	✓ both
deepseek-v3.1:671b	—	671 (MoE)	404	8.0	412	× too large

† ARM64 crash on Spark (Ollama bug); works on x86_64 Dell.

9.6.4 Thinking Models and Token Budgets

Several models in the inventory implement chain-of-thought reasoning internally, emitting a *thinking block* before their final answer. These include `qwen3` (all sizes), `deepseek-r1`, and `nemotron-3-super`. When calling these models via the OpenAI-compatible endpoint with a small `max_tokens` budget, the budget may be exhausted inside the thinking block, producing either an HTTP 500 error or an empty reply.

The correct mitigation is *not* to suppress thinking via the `/no_think` system prompt, which can leave the model with nothing to say outside the block. Instead, set `max_tokens` generously (500 or more) and strip thinking tags from the response:

```
# DeepSeek-R1 style:
<think> ... </think>

# Qwen3 / Ollama native style:
Thinking...
  ... reasoning text ...
...done thinking.
```

Listing 9.9: Thinking tag patterns to strip from model output

9.7 Configuring API Keys

You will receive an API key for OpenAI and Anthropic during our first session. You can get your own API for free at Groq. You need to create three API key variables following the procedure described below: `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GROQ_API_KEY`. Test the programs located at `resources/unit_7/FOO/` in the ELEVATE Academy repository: `agentGroq.py`, `agentGPT.py`, and `agentClaude.py`.

On Windows, execute the following three commands one at a time for each API key. The example for `OPENAI_API_KEY` is as follows:

```
setx OPENAI_API_KEY "[sk-... API key]"
Exit
echo %OPENAI_API_KEY%
```

In the code above, replace `[sk-... API key]` with your actual API key. The first command sets the environment variable permanently for your user account. The second command closes the Command Prompt to ensure the new environment variable is registered. The third command, run in a new Command Prompt window, prints the stored value to verify it was set correctly.

On Mac/Linux execute the following three commands one at a time for each API key. The example for `OPENAI_API_KEY` is as follows:

```
echo "export OPENAI_API_KEY='[sk-... API key]'" >> ~/.zshrc
source ~/.zshrc
echo $OPENAI_API_KEY
```

In the code above, replace `'[sk-... API key]'` with your actual API key.

As reported by Bryan Fowler, if you experience errors trying to add your environmental variables in a Mac/Linux, the most likely problem is ownership of the `~/.zshrc` file. In this case, execute

```
sudo chown $(whoami) ~/.zshrc
```

And try to create the environmental variables again.

As reported by Drew Stephen regarding Mac, "*apparently it doesn't work on the default c shell but switching to the zsh lets it run.*" You might need to close and reopen the terminal window from where you started for changes to become effective.

9.8 The FOO Framework and Agent Feasibility

9.8.1 Minimum Agent Count

The Max Multi-Agent Consensus System implements the FOO algorithm for Byzantine-fault-tolerant consensus. Given that any individual agent has an invalidation probability p, the minimum number of agents required to achieve consensus confidence C is:

$$N_{\text{foo}}(p, C) = \left\lceil \frac{\log(1 - C)}{\log(p)} \right\rceil \tag{9.5}$$

Selected values are given in Table 9.3.

Table 9.3: Minimum agent count N_{foo} by invalidation rate and confidence target

Per-agent invalidation rate p	Confidence C	N_{foo}
0.30	0.95	4
0.30	0.99	6
0.20	0.95	3
0.20	0.99	5
0.10	0.99	3
0.05	0.99	2

9.8.2 Agent Feasibility Score

The *Agent Feasibility Score* (AFS) couples the hardware budget to the FOO validity condition:

$$\text{AFS} = \frac{N_{\text{hw}}}{N_{\text{foo}}} \tag{9.6}$$

A configuration is *feasible* if and only if AFS ≥ 1.0. When AFS < 1.0, one or more of the following mitigations must be applied: reduce the context window (increases N_{hw}), use a smaller model (increases N_{hw}), or relax the confidence target C (reduces N_{foo}).

9.8.3 Reference Configurations

Dell VRAM-optimal (fast, no RAM spill). Five agents fitting entirely within 16 GB VRAM, yielding AFS $= 8.00$ at $p = 0.20$, $C = 0.99$:

Model	M_{agent} (GB)	Location
`gemma3:1b`	0.9	VRAM
`phi4-mini:3.8b`	2.8	VRAM
`gemma3:4b`	2.8	VRAM
`qwen3:4b`	2.8	VRAM
`mistral:7b`	4.5	VRAM
Total	13.8	(of 16 GB)

Spark full-capability configuration. Mixed-tier configuration using 110 GB unified memory:

Role	Model	M_{agent} (GB)	AFS
Orchestrator	`qwen3:32b-ctx8k`	19.3	1.67
Primary analyst	`llama3.3:70b-ctx8k`	42.5	0.67
Critic A	`mistral-small:22b`	14.0	2.33
Critic B	`phi4:14b`	8.8	4.00
Synthesiser	`mistral-nemo:12b`	7.7	4.67
Total		92.3	0.67

The minimum AFS of 0.67 (from `llama3.3:70b` alone in the pool) indicates that a full five-agent round cannot run simultaneously on the Spark with this model selection. The operational solution is to run the T1 models sequentially within a round and cache their outputs, or to replace `llama3.3:70b-ctx8k` with `qwen3:32b-ctx8k` to free memory for a fifth simultaneous agent.

9.9 Single-Machine Configuration (Dell)

In single-machine mode the Dell runs both the Alice application stack and the Ollama inference layer. This configuration is appropriate for prompt development and unit-level agent testing where network isolation is desirable.

9.9.1 Environment Variable

```
LLM_PROVIDER=ollama
LLM_BASE_URL=http://127.0.0.1:11434/v1
LLM_MODEL=mistral:7b
```

Listing 9.10: .env for single-machine mode

From WSL, the Windows Ollama instance is reachable via the WSL default gateway rather than localhost:

```
ip route | grep default | awk '{print $3}'
# Typical result: 172.24.0.1 (varies by WSL version and
  ↪ config)
```

Listing 9.11: Find the Windows gateway IP from WSL

```
LLM_PROVIDER=ollama
LLM_BASE_URL=http://172.24.0.1:11434/v1
LLM_MODEL=mistral:7b
```

Listing 9.12: .env for WSL accessing Windows Ollama

9.9.2 Model Selection for Single-Machine Development

The Dell's 16 GB VRAM constrains simultaneous model capacity. For development work targeting the FOO framework, the following tier assignment balances capability with VRAM residency:

- **Primary agent:** `deepseek-r1:8b` (5.5 GB), reasoning model with thinking mode; good proxy for larger consensus agents.
- **Fast critic:** `mistral-nemo:12b` (7.7 GB), tools-capable, fast, NVIDIA-trained.
- **Utility agent:** `gemma3:1b` (0.9 GB), sub-second responses for structured output tasks.

Total VRAM: 14.1 GB out of 16 GB. This leaves 1.9 GB headroom for the display server and background processes.

9.10 Two-Machine Configuration

In the two-machine setup the Dell hosts the application stack and the Spark serves all LLM inference. The Spark's 110 GB unified memory pool allows multiple large models to remain resident simultaneously, which is the hardware prerequisite for running a full FOO consensus round without sequential eviction.

9.10.1 Architecture

```
+----------------------------+   HTTP/REST    +----------------------------+
|        Dell 7770           | <------------- |       NVIDIA Spark         |
|                            |   port 11434   |                            |
|  Alice FastAPI             |                |  Ollama (port 11434)       |
|  PostgreSQL 16             |                |  systemd: ollama.service   |
|  Agents                    |                |  OLLAMA_HOST=0.0.0.0       |
|  Python venv: ALICE        |                |  OLLAMA_NUM_CTX=8192       |
|                            |                |                            |
|  .env:                     |                |  Models resident:          |
|    LLM_BASE_URL=           |                |    qwen3:32b-ctx8k         |
|    http://10.2.14.24:      |                |    llama3.3:70b-ctx8k      |
|    11434/v1                |                |    mistral-nemo:12b        |
+----------------------------+                +----------------------------+
```

9.10.2 Environment Variables

```
# Two-machine mode: Spark serves all inference
LLM_PROVIDER=ollama
LLM_BASE_URL=http://10.2.14.24:11434/v1
LLM_API_KEY=not-required

# Per-agent model assignment (Max routing policy)
AGENT_MODEL_ORCHESTRATOR=qwen3:32b-ctx8k
AGENT_MODEL_ANALYST=llama3.3:70b-ctx8k
AGENT_MODEL_CRITIC=mistral-nemo:12b
AGENT_MODEL_FAST=mistral:7b
```

Listing 9.13: .env for two-machine mode (Dell application host)

The `ProviderAdapter` protocol in Max makes switching between single-machine and two-machine modes a one-line change to `LLM_BASE_URL`. No application code changes are required.

9.10.3 Connectivity Verification

A connectivity test script (`connectivity_test.py`) validates the end-to-end path from the application host to the Ollama server. It tests four conditions in sequence, stopping at the first failure:

1. **TCP handshake:** confirms the host is reachable and the port is open.
2. **MAC ARP check:** informational only; confirms whether the machines are on the same Layer 2 segment. Failure here is expected on a routed university network and does not indicate a problem.
3. **Model list:** confirms Ollama is serving and returns the installed model list.
4. **Inference:** sends a minimal chat completion request and validates the response.

Machine profiles, ports, and test parameters are stored in `connectivity_config.json`, which is excluded from version control (`.gitignore`) because it contains environment-specific IP and MAC addresses. A `connectivity_config.json.template` with placeholder values is committed alongside the script.

9.11 Capacity Testing

A capacity test script (`capacity_test.py`) reads the model registry from `connectivity_config.json`, computes the theoretical N_{hw} and AFS for each model, and empirically validates each installed model by running a single inference call.

9.11.1 Platform Considerations

- **ARM64 (Spark):** `qwen3:4b` fails with an `exit status 2` crash in the Ollama ARM64 binary. This is an Ollama bug affecting this specific model and quantisation on ARM64. The same model works correctly on the x86_64 Dell. The script detects architecture at runtime and skips ARM64-specific failures appropriately.
- **Modelfile variants:** Context-bounded variants (e.g. `llama3.3:70b-ctx8k`) must be created separately on each machine. A model created via Modelfile on the Spark is not automatically available on the Dell. The script detects *base installed but variant missing* and emits the exact shell commands to create the variant, using PowerShell syntax on Windows and bash heredoc syntax on Linux.
- **VRAM vs. RAM:** On the Dell, the script distinguishes models that fit entirely in VRAM (fast inference) from those that spill to system RAM (significantly slower). Two configuration recommendations are produced: a VRAM-optimal configuration (fastest, may not reach N_{foo}) and a RAM-extended configuration (reaches N_{foo} if RAM permits, but with a performance warning for spilled models).

9.11.2 Ollama CLI Reference

```
# List installed models
ollama list

# Show currently loaded models and memory usage
ollama ps

# Pull a model from the registry
ollama pull qwen3:32b

# Interactive chat session
ollama run mistral:7b

# One-shot prompt (no interactive session)
ollama run mistral:7b "Explain the rhizome model in one
   ↪ paragraph"

# Remove a model
ollama rm mistral:7b

# Test the OpenAI-compatible API directly
curl http://localhost:11434/v1/models
curl http://localhost:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"mistral:7b",
```

```
24        "messages":[{"role":"user","content":"Hello"}]}'
```

Listing 9.14: Essential Ollama CLI commands

9.12 Deployment Parity

A core principle of this infrastructure is that the development environment mirrors production without containerisation. Both machines run native Ubuntu (or WSL2 for the Windows host), Ollama is managed by `systemd`, and configuration is injected via environment files rather than code changes.

The two-tier configuration model is:

- **Development:** `.env` file in the project root, loaded by `python-dotenv` with `override=False`.
- **Production:** `systemd` `EnvironmentFile=` directive pointing to a file outside the repository. No code changes required at deployment.

Switching from single-machine to two-machine mode, or from the Dell's local Ollama to the Spark, requires only updating `LLM_BASE_URL` in the environment file. The `ProviderAdapter` protocol transparently routes requests to whichever endpoint is configured.

9.13 Setting Up Ollama with a GUI on Ubuntu

To transition from a command-line interface to a graphical experience, follow these steps to install the Ollama backend and the Open WebUI frontend.

9.13.1 Step 1: Install Ollama Backend

First, install the core Ollama service using the official installation script:

```
curl -fsSL https://ollama.com | sh
```

9.13.2 Step 2: Deploy Open WebUI via Docker

Open WebUI is the most popular ChatGPT-like interface for Ollama. Run the following Docker command to pull the image and start the container:

```
docker run -d -p 3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -v open-webui:/app/backend/data \
  --name open-webui \
  ghcr.io/open-webui/open-webui:main
```

9.13.3 Step 3: Access the Interface

Once the container is running, open your preferred web browser and navigate to:

`http://localhost:3000`

You will be prompted to create an admin account (stored locally) to begin chatting with your models.

9.13.4 Technical Considerations

- **GPU Support:** For NVIDIA users, ensure the `nvidia-container-toolkit` is installed to enable hardware acceleration within Docker.
- **External Access:** To access this GUI from another machine, set the environment variable `OLLAMA_HOST=0.0.0.0` in your systemd service configuration.

Chapter 10

Assessment and Progress Record

This chapter is a fillable PDF form intended for faculty and staff who are onboarding as contributors to the Department of Mathematics curriculum repository. Each section corresponds to one category of readiness: Required, Essential, Desirable, and Optional. Checkboxes record binary completion of verification steps; text fields record short written responses that demonstrate conceptual understanding. Use this chapter as a running checklist during onboarding.

Saving your responses requires Adobe Acrobat Reader (free for Windows, macOS, and Linux; available at `https://get.adobe.com/reader/`). Other viewers behave as follows:

- **macOS Preview:** displays form fields and allows typing in them, but *does not save the entered data*. If you close and reopen the file in Preview, your responses will be lost.
- **Google Chrome (built-in PDF viewer):** displays and accepts input in form fields, but *does not save the data*. Printing to PDF from Chrome will not preserve field contents either.
- **Most Linux PDF viewers** (Evince, Okular, Zathura): *ignore form fields entirely*. Fields will not appear interactive.
- **Adobe Acrobat Reader:** fully saves, prints, and exports form data. Use **File** > **Save** (not Save As) after completing each section.

How to interpret the four categories:

Required Core repository tasks. These must be completed before a contributor can make any changes to the curriculum document.

Essential Local environment setup: LaTeX, Python, Git, and Visual Studio Code. These must be working correctly before beginning any authoring or scripting work.

Desirable Conceptual understanding of the tools introduced in the infrastructure chapters. Completing these demonstrates readiness to work independently on the repository.

Optional Local LLM infrastructure (Ollama). Not required for curriculum authoring; relevant for contributors who wish to use local language models in their workflow.

10.1 Required: Repository Onboarding

The following items verify that you can retrieve, compile, and contribute to the curriculum repository. All items must be completed before editing any source file.

☐ **R1.** `git clone` of the repository completes without errors and the local working copy contains the `curriculum/` folder. (See Chapter 8, Section 8.2.7.)

☐ **R2.** Running `pdflatex MathCurriculum.tex` from inside the `curriculum/` folder produces `MathCurriculum.pdf` with no fatal errors. (See Chapter 7, Section 7.1.)

☐ **R3.** Running `gh` (or `gh.bat` on Windows) with a commit message successfully stages, commits, and pushes a change to the repository, and `VERSION.txt` is updated with the new hash. (See Chapter 8.)

☐ **R4.** The commit hash in `VERSION.txt` matches the value returned by `git rev-parse --short HEAD` in your local repository. (See Chapter 8.)

R5. Describe the two-part structure of the curriculum document. What is contained in the infrastructure chapters, and what is contained in the course-unit chapters? How are unit chapters identified in the source tree? (See the Introduction.)

10.2 Essential: Environment and Toolchain Setup

Check each box when the corresponding verification step succeeds. All items in this section should be checked before beginning any authoring or scripting work on the repository.

☐ **E1.** `pdflatex --version` returns a version number in the terminal. (See Chapter 2.)

☐ **E2.** `python --version` returns a version number in the terminal. (See Chapter 2, Section 2.1.)

☐ **E3.** `git --version` returns a version number. (See Chapter 2, Section 2.6.)

☐ **E4.** `code .` opens Visual Studio Code from a terminal window. (See Chapter 2, Section 2.8.)

☐ **E5.** The Python virtual environment activates without errors, and the packages listed in `pyproject.toml` (or `requirements.txt`) import cleanly in that environment. (See Chapter 5, Section 5.1.4.)

☐ **E6.** Running `latexmk -pdf MathCurriculum.tex` produces a correctly paginated PDF with a table of contents and all cross-references resolved. (See Chapter 7.)

10.3 Desirable: Technical Understanding

These items demonstrate working understanding of the tools introduced in the infrastructure chapters. Written responses need not be long; two or three sentences each are sufficient.

D1. What is the current working directory when you first open a terminal? Write the command you would use to navigate to the `curriculum/` folder inside the cloned repository. (See Chapter 3, Section 3.1.)

D2. What is the difference between a Python virtual environment and the global Python installation? Give one scenario in the context of this repository where you would need a separate virtual environment. (See Chapter 5.)

D3. Why does `git push` require a prior `git commit`? What is the conceptual difference between the two operations, and why does the `gh.bat` script perform both in sequence? (See Chapter 8.)

D4. In LaTeX, what is the practical difference between `\[...\]` and `$$...$$`? Which form should be used in new curriculum source files, and why? (See Chapter 7, Section 7.4.)

D5. What does the `regen.py` script do when run for a given course unit? Describe the input it reads, the output it produces, and the role of the `ASSESSMENTS` dictionary in controlling the output. (See the repository `Script/` folder and associated documentation.)

D6. Explain the label scheme used for cross-references in the curriculum source files. What is the format of a section label, how is the four-character mnemonic derived, and how is the hash suffix computed? (See the curriculum pipeline documentation.)

10.4 Optional: Local LLM Infrastructure (Ollama)

These items apply only to contributors who install and configure the local Ollama LLM infrastructure. They are not required for curriculum authoring or repository maintenance.

☐ **O1.** Ollama is installed and `ollama list` returns at least one model in the terminal. (See Chapter 9, Section 9.4.)

☐ **O2.** `connectivity_test.py` passes all verification checks (TCP handshake, model list, inference). (See Chapter 9.)

O3. Explain the tradeoff between context window size and memory consumption in Ollama. Why is `OLLAMA_NUM_CTX=8192` a safe default for large models on hardware with limited VRAM? (See Chapter 9, Section 9.6.)